

Discrete Mathematics

David M. Clark

W. Ted Mahavier

SUNY New Paltz

Lamar University

(Chapter 3: W.T. Mahavier < E.L. May < K.M. Shannon¹)

¹If each of X and Y is an author, then the relation $X < Y$ indicates that the notes were originally authored by Y and subsequently modified by X, who takes full responsibility for the current version. While Y is credited with the genesis of the notes, s/he makes no claim to the accuracy of the current version which may or may not reflect her/his original vision.

Contents

To the Student	iv
1 Sets	1
1.1 Project: More on $\oplus$ and $\times$	5
2 Functions	6
2.1 Project: More on Inverses, Composition and Relations	10
3 Counting	12
3.1 Project: Unordered Samples and Probability, Inclusion and Exclusion	19
4 Induction and Recursion	22
4.1 Project: More Induction	26
5 Euler Paths and Circuits	29
5.1 Project: Proof of Fleury's Theorem	42
6 Traveling Salesman Problems	44
6.1 Project: Edges with Equal Weights	49
7 Spanning Trees	51
7.1 Projects: Tree Equivalence and Kruskal's Theorem	55
8 Equivalence Relations and Partitions	58
8.1 Project: More on Equivalence Classes	62
9 Finite State Machines	63
9.1 Finite State Transducers	64
9.2 Finite State Acceptors	66
9.3 Regular Expressions	70

9.4 Project: Reversed Strings and Palindromes	71
10 Machine Minimization	73

To the Student

How this class works

Every semester I collect written evaluations from my classes and the students who have taken this course have been overwhelmingly positive in their comments. While challenged by the material and the method, they enjoyed the class. If you'd like, I'll read random comments from the last semester I taught it. I'll print them out, let you randomly select a number and I'll read that comment.

The reason I open with a discussion of my evaluations, is because this class will be taught in a way that is (most likely) different from any mathematics classes you have encountered in the past. Most of the class will be devoted to students working problems at the board and half of your grade will be determined by the amount of mathematics that you produce in this class. I use the word produce because it is my belief that the best way to learn mathematics is by doing mathematics. Therefore, just as I learned to ride a bike by getting on and falling off, I expect that you will learn mathematics by attempting it and (occasionally) falling off! You will have a set of notes (you are reading them now) that you will turn into a book by working through the problems. I urge you to seriously consider the value of becoming an independent thinker who tackles doing mathematics (and everything else in life) on your own, rather than waiting for someone else to show you how to do things.

A common pitfall

There are two ways in which students often approach my classes. The first is to say, "I'll wait and see how this works and then see if I like it and put some problems up later in the semester after I catch on." Think of the course as a forty yard dash. Do you really want to wait and see how fast the other runners are before you start running? If you try to do the problems every night, then either you will get a problem (YEA!) and be able to put it on the board or you will struggle with the problem and learn a lot in your struggle. If you have worked hard on a problem, you may go to the board

with your work and show us what you tried. Doing so will almost surely give you further ideas on how to solve it. If someone else puts the problem on the board then you will be able to ask questions and help yourself and others understand it, which also counts toward your presentation grade. And you will be able to say to yourself, “Ahhhh, now I see where I went wrong and now I can do this one and a few more for next class.” If you do not try problems each night, then you will watch the student put the problem on the board, but perhaps will not quite catch all the details and when you study for the tests or try the next problems you will have only a loose idea of how to tackle such problems. Basically, you have seen it only once in this case. The first student saw it once when s/he tackled it on his own, again when either s/he put it on the board or another student presented it, and a third time when s/he studied for the next test or quiz. Hence the difference between these two approaches is the difference between participating and watching a movie. I hope that each of you will tackle this course with the attitude that you will learn this material and thus will both enjoy and benefit from the class.

Boardwork

Because the board work constitutes half of your grade, let's put your mind at ease regarding this part of the class. First, by coming to class everyday you will earn a 60% on board work. Every problem you present pushes that grade higher. You may come see me anytime for an indication of what grade your current level of participation will earn you at the end of the semester. And I give one presentation credit for an office visit within the first two weeks of the semester. Here are some rules and guidelines associated with the board work.

1. Your only sources are to be me and these notes.
2. You are not allowed external materials (web, books, etc.).
3. You are not allowed to work with or ask questions of other individuals (friends, classmates, teachers, etc.).
4. You may not present a problem that you have seen in another class.
5. If there is a problem that you have spent a lot of time on and you don't want to see a solution to it yet, you may step out of the room while it is presented and turn it in for some credit.

I will call for volunteers every day and will pick the person with the least presentations to present a given problem. You may inform me that you have a problem in advance (which I appreciate), but the problem still goes to the person with the least presentations on the day I call for solutions. Ties

are broken either randomly (at the beginning) or by test grades (lower test grades taking priority). A student who has not gone to the board on a given day will be given precedence over a student who has gone to the board that day. To “present” a problem at the board means to have written the problem statement up, to have written a correct solution using complete mathematical sentences, and to have answered all students’ questions regarding the problem. When you have worked hard on a problem, even if you aren’t sure your work is correct, *go to the board*. Going to the board can only help your grade. Since you will be communicating with other students in class on a regular basis, here are several guidelines that will help you. Remember that the whole class is on your side because everyone wants to see the problem presented correctly and to understand it.

When you speak, don’t use the words “obvious,” “stupid,” or “trivial.” Don’t attack anyone personally or try to intimidate anyone. Don’t get mad or upset at anyone, even yourself. And if you do, try to get over it quickly. Don’t be upset when you make a mistake - brush it off and learn from it. Don’t let anything go on the board that you don’t fully understand. Don’t say to yourself, “I’ll figure this out at home.” Don’t use concepts we have not defined. Don’t try to put up a problem you have not written up.

Do prepare arguments in advance. Do be polite and respectful. Do learn from your mistakes. Do ask questions such as, “Can you tell me how you got the third line?” Do let people answer when they are asked a question. Do refer to earlier results and definitions by number when possible. For example, “by Definition 29” we know that...

How to study

1. Read over your notes from class *every* day.
2. Make a list of questions to ask at the beginning of the next class.
3. Review the recent problems.
4. Work on *several* new problems.
5. Write up as many solutions as you can so that you can simply copy your solutions onto the board the next day.

Some problems are hard. If you don’t get one, don’t give up on it. Move on to another problem and come back to that problem later. The problems worth solving in life are not solved in five minutes.

What is discrete mathematics?

The word “discrete” means finite in this setting. Thus, discrete mathematics means the study of the topics in mathematics that can be approached

without using infinite sets. Even so, you'll see a few infinite sets in the course.

W. Ted Mahavier
January 2015

Chapter 1

Sets

Every mathematics, science, and engineering course uses sets, the basic building blocks of mathematics, so we start here. It might seem that we should start with numbers. However, set theory is required to do a mathematically rigorous development of the numbers as well, so sets are the best starting point.

We won't define the words "element," "set" and "universe." Rather, we will rely on our intuition. We will consider a **set** to be a collection of **elements** coming from some **universe** $\mathbb{U}$ of elements. For example, if we are talking about numbers, our universe might be the set of all real numbers or might be the set of all integers. If we are talking about animals, our universe might be all animals in the San Diego Zoo or it might be all animals on this planet.

Axiom 1. Containment. *If $\mathbf{A}$ is a set, then for each element x in $\mathbb{U}$ either x is in $\mathbf{A}$ (written $x \in \mathbf{A}$) or x is not in $\mathbf{A}$ (written $x \notin \mathbf{A}$).*

Definition 1. *If $\mathbf{A}$ and $\mathbf{B}$ are sets and every element of $\mathbf{A}$ is also in $\mathbf{B}$, then we say that $\mathbf{A}$ is a **subset** of $\mathbf{B}$ and write $\mathbf{A} \subseteq \mathbf{B}$.*

Axiom 2. Set Equality. *If $\mathbf{A}$ and $\mathbf{B}$ are sets that have the same elements (that is, $\mathbf{A} \subseteq \mathbf{B}$ and $\mathbf{B} \subseteq \mathbf{A}$), then we say that $\mathbf{A}$ and $\mathbf{B}$ are the same set and write $\mathbf{A} = \mathbf{B}$.*

Axiom 3. Set Specification. *If $\mathbf{P}$ is a property that each element in $\mathbb{U}$ either has or does not have, then there is a set, denoted by*

$$\{x \in \mathbb{U} \mid x \text{ has property } \mathbf{P}\},$$

whose elements are exactly those elements in $\mathbb{U}$ having property $\mathbf{P}$.

Definition 2. *The set $\mathbb{R}$ is the set of all real numbers.*

Example 1. *Are the sets $\mathbf{A} = \{(x, y) \mid x \in \mathbb{R} \text{ and } x = \sqrt{y}\}$ and $\mathbf{B} = \{(x, y) \mid x \in \mathbb{R} \text{ and } x^2 = y\}$ the same sets? What are some elements of $\mathbf{A}$? Of $\mathbf{B}$?*

Definition 3. The *empty set* is the set having no members,

$$\emptyset = \{x \in \mathbb{U} \mid x \neq x\}.$$

We can construct new sets from old using the Set Specification Axiom.

Definition 4. Let **A** and **B** be sets. The *intersection* of **A** and **B** is the set

$$\mathbf{A} \cap \mathbf{B} = \{x \in \mathbb{U} \mid x \in \mathbf{A} \text{ and } x \in \mathbf{B}\}.$$

Definition 5. Let **A** and **B** be sets. The *union* of **A** and **B** is the set

$$\mathbf{A} \cup \mathbf{B} = \{x \in \mathbb{U} \mid x \in \mathbf{A} \text{ or } x \in \mathbf{B}\}.$$

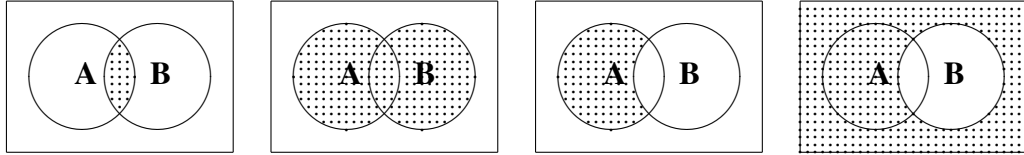
Definition 6. Let **B** be a set. The *complement* of **B** is the set

$$\sim \mathbf{B} = \{x \in \mathbb{U} \mid x \notin \mathbf{B}\}.$$

Definition 7. Let **A** and **B** be sets. The *difference* between **A** and **B** is the set

$$\mathbf{A} \sim \mathbf{B} = \mathbf{A} \cap (\sim \mathbf{B}).$$

The following illustrations are **Venn diagrams** for the sets just defined.



$\mathbf{A} \cap \mathbf{B}$

$\mathbf{A} \cup \mathbf{B}$

$\mathbf{A} \sim \mathbf{B}$

$\sim \mathbf{B}$

Different expressions might represent the same set as illustrated by the next example, which together with Problem 1 will show that

$$\mathbf{A} \cap (\mathbf{B} \cup \mathbf{C}) = (\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C}).$$

Example 2. For every choice of sets **A**, **B** and **C**,

$$\mathbf{A} \cap (\mathbf{B} \cup \mathbf{C}) \subseteq (\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C}).$$

For Problems 1 – 19 we will assume that **A**, **B**, and **C** are sets of elements from some universe $\mathbb{U}$.

Problem 1. Show that $(\mathbf{A} \cap \mathbf{B}) \cup (\mathbf{A} \cap \mathbf{C}) \subseteq \mathbf{A} \cap (\mathbf{B} \cup \mathbf{C})$.

You are already familiar with operations on numbers such as addition, subtraction, multiplication, and division ($+$, $-$, $*$, $\div$). We have introduced the operations on sets such as intersection, union, and difference ($\cap$, $\cup$, and $\sim$). Soon, we will introduce more set operations, including $\times$, $\oplus$ and $\circ$. All of these number and set operations are referred to as **binary operations** because each operation takes two inputs.

For the next few problems, illustrate each of the following identities with Venn Diagrams *and* write down a proof using the Set Equality Axiom.

Problem 2. *The Commutative Laws*

1. $\mathbf{A \cap B = B \cap A}$

2. $\mathbf{A \cup B = B \cup A}$

Problem 3. *The Associative Laws*

1. $\mathbf{A \cap (B \cap C) = (A \cap B) \cap C}$

2. $\mathbf{A \cup (B \cup C) = (A \cup B) \cup C}$

Problem 4. *Ask one question that you have after reading the introduction.*

Problem 5. *The Distributive Laws*

1. $\mathbf{A \cup (B \cap C) = (A \cup B) \cap (A \cup C)}$

2. $\mathbf{A \cap (B \cup C) = (A \cap B) \cup (A \cap C)}$

Problem 6. *The Absorption Laws*

1. $\mathbf{A \cup (A \cap B) = A}$

2. $\mathbf{A \cap (A \cup B) = A}$

For the next problem, it is helpful to make the observation about the empty set that *any* statement starting with “If $x \in \emptyset$, then ...” is a true statement. For example, the statement “If $x \in \emptyset$, then x is a seven-headed dog.” is a true statement. Why? Because the only way that statement could be false would be to have some value of x such that $x \in \emptyset$ but x was not a seven-headed dog. But there is no such x since there is nothing in the empty set.

Problem 7. *The Identity Laws*

1. $\mathbf{A \cup \emptyset = A}$

2. $\mathbf{A \cap \mathbb{U} = A}$

Problem 8. *The Inverse Laws*

1. $A \cup \sim A = U$
2. $A \cap \sim A = \emptyset$

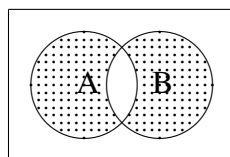
Problem 9. *DeMorgan's Laws*

1. $\sim (A \cap B) = \sim A \cup \sim B$
2. $\sim (A \cup B) = \sim A \cap \sim B$

Definition 8. The *symmetric difference* between sets **A** and **B** is defined by

$$A \oplus B = (A \cup B) \sim (A \cap B).$$

We illustrate $A \oplus B$ via a Venn diagram.



Problem 10. $A \oplus B = (A \sim B) \cup (B \sim A)$

Problem 11. $A \oplus (B \oplus C) = (A \oplus B) \oplus C$

Example 3. *Does*

$$A \oplus (B \cup C) = (A \oplus B) \cup (A \oplus C)?$$

Restated, does $\oplus$ distribute over $\cup$?

It is likely that you “know” what an ordered pair is from previous courses, but unlikely that you have ever seen a precise definition. The next two problems make precise the notion of ordered pairs.

Problem 12. Suppose that each of a , b , c and d is an element. For each statement, determine if it is true or false.

1. If $\{a, b\} = \{c, d\}$, then $a = c$ and $b = d$.
2. If $a = c$ and $b = d$, then $\{a, b\} = \{c, d\}$.

Definition 9. If each of a and b is an element, then by the **ordered pair** (a, b) we mean the set $\{\{a\}, \{a, b\}\}$. The elements a and b are known as the first and second coordinates of (a, b) , respectively.

Problem 12 hinted at the property that we want from our new definition for ordered pairs. We want that $(a, b) = (c, d)$ if and only if $a = c$ and $b = d$. The next problem allows you to show that our definition for ordered pairs, $(a, b) = \{\{a\}, \{a, b\}\}$, satisfies this property and therefore is a valid definition.

Problem 13. Suppose each of a , b , c and d is an element. Use Definition 9 to prove the following two statements:

1. If $(a, b) = (c, d)$, then $a = c$ and $b = d$.

2. If $a = c$ and $b = d$, then $(a, b) = (c, d)$.

Definition 10. If $\mathbf{A}$ and $\mathbf{B}$ are sets, the **Cartesian product** of $\mathbf{A}$ and $\mathbf{B}$, denoted by $\mathbf{A} \times \mathbf{B}$, consists of all ordered pairs (x, y) where $x \in \mathbf{A}$ and $y \in \mathbf{B}$. Restated,

$$\mathbf{A} \times \mathbf{B} = \{ (x, y) \mid x \in \mathbf{A} \text{ and } y \in \mathbf{B} \}.$$

Problem 14. Prove or give a counter-example:

$$\mathbf{A} \cap (\mathbf{B} \times \mathbf{C}) = (\mathbf{A} \cap \mathbf{B}) \times (\mathbf{A} \cap \mathbf{C})$$

1.1 Project: More on $\oplus$ and $\times$

For the following statements, either prove that it is true using the set equality axiom, or prove that it is false by exhibiting a counterexample.

Problem 15. $\mathbf{A} \oplus (\mathbf{B} \cap \mathbf{C}) = (\mathbf{A} \oplus \mathbf{B}) \cap (\mathbf{A} \oplus \mathbf{C})$

Problem 16. $\mathbf{A} \cap (\mathbf{B} \oplus \mathbf{C}) = (\mathbf{A} \cap \mathbf{B}) \oplus (\mathbf{A} \cap \mathbf{C})$

Problem 17. $\sim (\mathbf{A} \oplus \mathbf{B}) = \sim \mathbf{A} \oplus \sim \mathbf{B}$

Problem 18. $\mathbf{A} \oplus \mathbf{B} = \sim \mathbf{A} \oplus \sim \mathbf{B}$

Problem 19. $\mathbf{A} \cup (\mathbf{B} \oplus \mathbf{C}) = (\mathbf{A} \cup \mathbf{B}) \oplus (\mathbf{A} \cup \mathbf{C})$

Problem 20. $\mathbf{A} \times (\mathbf{B} \cup \mathbf{C}) = (\mathbf{A} \times \mathbf{B}) \cup (\mathbf{A} \times \mathbf{C})$

Problem 21. $\mathbf{A} \cup (\mathbf{B} \times \mathbf{C}) = (\mathbf{A} \cup \mathbf{B}) \times (\mathbf{A} \cup \mathbf{C})$

Chapter 2

Functions

Every mathematics, science, and engineering course makes heavy use of functions. Having developed a deep understanding of sets and ordered pairs, we are now in a position to define function in terms of set of ordered pairs.

First, recall from Definition 10 that the Cartesian product of $\mathbf{X}$ and $\mathbf{Y}$ is $\mathbf{X} \times \mathbf{Y} = \{(x, y) \mid x \in X \text{ and } y \in Y\}$.

Definition 11. Let $\mathbf{X}$ and $\mathbf{Y}$ be sets. A **relation** on $\mathbf{X} \times \mathbf{Y}$ is any subset of $\mathbf{X} \times \mathbf{Y}$. A **function** on $\mathbf{X} \times \mathbf{Y}$ is a relation on $\mathbf{X} \times \mathbf{Y}$ where no two elements have the same first coordinates. The set of all first coordinates of a relation is called the **domain** and the set of all second coordinates of a relation is called the **range**.

Example 4. Let $\mathbf{A} = \{x \in \mathbb{R} \mid -3 \leq x \leq 3\}$. The set $\mathbf{C} = \{(x, y) \in \mathbb{R} \times \mathbb{R} \mid x^2 + y^2 = 9\}$ is a relation on $\mathbf{A} \times \mathbf{A}$ but not a function on $\mathbf{A} \times \mathbf{A}$.

Example 5. The set $f = \{(x, y) \mid x \in \mathbb{R} \text{ and } y = 2x - 3\}$ is a function on $\mathbb{R} \times \mathbb{R}$. How do we prove this?

Problem 22. Let $\mathbf{A} = \{1, 2, 3\}$ and $\mathbf{B} = \{\square, \diamond, \triangle\}$. Which of the following are relations on $\mathbf{A} \times \mathbf{B}$? Which are functions

1. $\{(1, \square), (1, \triangle), (2, \diamond)\}$
2. $\{(3, \square), (1, \triangle), (2, \triangle)\}$
3. $\{((1, 1), \square), ((1, 2), \triangle), ((2, 1), \diamond), ((2, 2), \diamond)\}$

Problem 23. Consider the function f defined by

$$f = \{(x, y) \mid x \in \mathbb{R} \sim \{1\} \text{ and } y = \frac{2x}{x-1}\}.$$

State the domain and the range of this function.

Problem 24. Consider

$f = \{(x, y) \in \mathbb{R} \times \mathbb{R} \mid x \in \mathbb{R} \text{ and } y = \sin(2x) + \sin^2(x) + \cos^2(x)\}$ and $g = \{(x, y) \mid x \in \mathbb{R} \text{ and } y = 2 \sin(x) \cos(x) + 1\}$.

Are these the same function?

Since we often think of a function f on $\mathbf{X} \times \mathbf{Y}$ as a rule assigning elements of $\mathbf{X}$ to elements of $\mathbf{Y}$, we often write $f : \mathbf{X} \rightarrow \mathbf{Y}$. When (x, y) is an element of f we write, $f(x) = y$ and say that f **maps** x to y . When we use this notation, it means that $\mathbf{X}$ is the domain of f and $\mathbf{Y}$ contains the range of f . In such cases, we call $\mathbf{Y}$ the **codomain** of f .

Definition 12. If $f : \mathbf{X} \rightarrow \mathbf{Y}$ is a function, then f is **one-to-one** if no two elements of f have the same second coordinate and different first coordinates. Restated, no two elements of $\mathbf{X}$ can map to the same element of $\mathbf{Y}$. We say that f is **onto the set $\mathbf{Y}$** if for each element $y \in \mathbf{Y}$ there is some element $x \in \mathbf{X}$ such that $f(x) = y$. A function $f : X \rightarrow Y$ that is both one-to-one and onto is a **bijection**.

Example 6. Let $f = \{(x, y) \mid x \in \mathbb{R} \text{ and } y = x^2 + 3\}$. We might write the same function as $f : \mathbb{R} \rightarrow \mathbb{R}$ where $f(x) = x^2 + 3$. Is f onto $\mathbb{R}$? Is f onto $\{y \mid y \geq 3\}$? We would call $\mathbb{R}$ a codomain of f .

Problem 25. Suppose $f(x) = \frac{x^2 - 5x + 6}{x - 3}$ and $g(x) = x - 2$. What is the largest subset of $\mathbb{R}$ that is an allowable domain for f ? For g ? Does $f = g$?

Problem 26. Let $\mathcal{P}$ denote the set of all subsets of a set $\mathbb{U}$, and choose two particular sets $\mathbf{A}, \mathbf{B} \in \mathcal{P}$. Let $f : \mathcal{P} \rightarrow \mathcal{P}$ and $g : \mathcal{P} \rightarrow \mathcal{P}$ be the functions defined by: $f(\mathbf{X}) = \mathbf{X} \cap (\mathbf{A} \sim \mathbf{B})$ for every $\mathbf{X} \in \mathcal{P}$ and $g(\mathbf{X}) = (\mathbf{X} \cap \mathbf{A}) \sim (\mathbf{X} \cap \mathbf{B})$ for every $\mathbf{X} \in \mathcal{P}$. What is the domain of f ? Of g ? Does $f = g$?

Definition 13. For real valued functions $f : \mathbb{R} \rightarrow \mathbb{R}$ and $g : \mathbb{R} \rightarrow \mathbb{R}$ define a new function $f + g : \mathbb{R} \rightarrow \mathbb{R}$, called the **sum** of f and g , by the rule $(f + g)(x) = f(x) + g(x)$ for $x \in \mathbb{R}$.

Problem 27. Suppose each of $f : \mathbb{R} \rightarrow \mathbb{R}$ and $g : \mathbb{R} \rightarrow \mathbb{R}$ are functions and prove that $f + g = g + f$ by showing that $(f + g)(x) = (g + f)(x)$ for all $x \in \mathbb{R}$. This shows that addition of functions is commutative.

Problem 28. Let $\mathbf{A} = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$. Table 2.1 defines a function $f : \mathbf{A} \rightarrow \mathbf{A}$. For each $x \in \mathbf{A}$, the value of $f(x)$ is written below x . Is f one-to-one? Is f onto $\mathbf{A}$? Is f a bijection?

x	1	2	3	4	5	6	7	8	9
f(x)	5	7	9	3	1	2	6	4	8

Table 2.1: A function, f

Problem 29. Let $f = \{(x, y) \mid x \in \mathbb{R} \sim \{2\} \text{ and } y = \frac{x}{x-2}\}$.

1. Is f a one-to-one function?

2. Is f onto the set $\mathbb{R}$?

Problem 30. Define $f : \mathbb{R} \rightarrow \mathbb{R}$ by $f(x) = \sqrt[3]{x-1}$.

1. Is f a one-to-one function?

2. Is f onto $\mathbb{R}$?

By Definition 11, every function $f : \mathbf{X} \rightarrow \mathbf{Y}$ is onto its range since the range of f is the set of all y such that $(x, y) \in f$ for some $x \in \mathbf{X}$. Restated, the range of f is $\{y \mid (x, y) \in f\} = \{f(x) \mid x \in \mathbf{X}\}$.

Should you ever board an airplane (a function) that “maps” you from Houston to Chicago, at some point in the future you will definitely want to board another airplane (the inverse function) that “maps” you back home! Countless people have been lost because they built a time travel machine but forgot to build the inverse time machine!

Problem 31. Figure 2.1 shows the relationship between the Fahrenheit and Celsius temperature scales. Write a formula (equation) for a function f that converts Celsius to Fahrenheit, and a function c that converts Fahrenheit back to Celsius. Verify that $f(c(F)) = F$ for every $F \in \mathbb{R}$ and that $c(f(C)) = C$ for every $C \in \mathbb{R}$.

Definition 14. Given a function $f : X \rightarrow Y$, the relation f^{-1} is defined by $f^{-1} = \{(y, x) \mid (x, y) \in f\}$.

The set f^{-1} might not be a function. The next two problems tell us exactly when f^{-1} is a function.

Problem 32. Let f be a function from X onto Y . Show that if f is one-to-one, then f^{-1} is a function.

Problem 33. Let f be a function from X onto Y . Show that if f^{-1} is a function, then f is one-to-one.

Together Problems 32 and 33 show that a function f is one-to-one if and only if f^{-1} is a function. Many mathematicians do not define the inverse of a relation and don't define the inverse of a function unless that function is one-to-one. In this case they would say “if f is a function that is one-to-one, then there is a function f^{-1} so that $f(f^{-1}(x)) = x = f^{-1}(f(x))$ ”.

Problem 34. For each function f determine if f^{-1} is a function.

1. $f = \{(x, y) \mid x \in \mathbb{R} \text{ and } y = x(x-1)\}$

2. $f : \mathbb{R} \rightarrow \mathbb{R}$ defined by $f(x) = \sqrt[3]{x-1}$

Problem 35. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ with $f(x) = 3x - 7$.

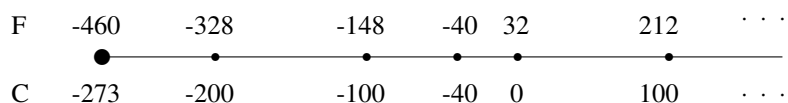


Figure 2.1: Thermometer showing both Fahrenheit and Celsius temperature scales

1. Prove that f is onto $\mathbb{R}$.
2. Prove that f is one-to-one.
3. Find a formula for $f^{-1}(y)$.
4. Verify that for all $x \in \mathbb{R}$ we have, $f^{-1}(f(x)) = x$ and $f(f^{-1}(x)) = x$.

Problem 36. Show that every non-constant linear function $l(x) = mx + b$, with $m \neq 0$, is a bijection $l : \mathbb{R} \rightarrow \mathbb{R}$. Find a formula for $l^{-1}(y)$.

The binary operations of addition and multiplication can be used to combine any two numbers in order to get a new number. The binary operations of intersection, union, difference and symmetric difference can be used to combine two sets in order to form a new set. Addition, subtraction, multiplication and division of real-valued functions are also binary operations. Another binary operation on functions is called composition.

Definition 15. Suppose that $g : \mathbf{A} \rightarrow \mathbf{B}$ and $f : \mathbf{B} \rightarrow \mathbf{C}$. We define the **composition** of f and g (denoted by $f \circ g$) to be the function from $\mathbf{A}$ to $\mathbf{C}$ satisfying

$$(f \circ g)(x) = f(g(x)).$$

In the Table 2.2 below, we have four bijections from $\{1, 2, 3, 4\}$ onto $\{1, 2, 3, 4\}$. The first bijection, i , is the **identity map** since it maps $1 \rightarrow 1$, $2 \rightarrow 2$, $3 \rightarrow 3$ and $4 \rightarrow 4$.

i	p	q	r
1234	1234	1234	1234
1234	2134	1243	2143

Table 2.2: Bijections

Let's compute $q \circ r$.

1. $(q \circ r)(1) = q(r(1)) = q(2) = 2 = p(1)$,
2. $(q \circ r)(2) = q(r(2)) = q(1) = 1 = p(2)$,
3. $(q \circ r)(3) = q(r(3)) = q(4) = 3 = p(3)$, and

$$4. (q \circ r)(4) = q(r(4)) = q(3) = 4 = p(4).$$

Therefore, $q \circ r = p$.

Problem 37. Use the example and Table 2.2 to fill in the rest of Table 2.3.

$\circ$	i	p	q	r
i				
p				
q				p
r				

Table 2.3: Composition

Problem 38. What is this inverse of each of these bijections?

Problem 39. Addition of numbers is **commutative** because for any two numbers a and b we have $a + b = b + a$. If for every set $\mathbf{A}$, every $f : \mathbf{A} \rightarrow \mathbf{A}$ and every $g : \mathbf{A} \rightarrow \mathbf{A}$ it is true that

$$f \circ g = g \circ f,$$

then we would say that composition of functions is also **commutative**. Either show that composition is commutative or give a counter example by finding two functions for which the statement is not true.

Problem 40. Intersection of sets is **associative** since for any sets $A, B,$ and C we have $\mathbf{A} \cap (\mathbf{B} \cap \mathbf{C}) = (\mathbf{A} \cap \mathbf{B}) \cap \mathbf{C}$. Is composition of functions associative? Restated, is it true that if each of f, g and h is a function, then $(f \circ g) \circ h = f \circ (g \circ h)$?

2.1 Project: More on Inverses, Composition and Relations

Problem 41. 1. Is Problem 13 valid if we define $(a, b) = \{\{a, 1\}, \{2, b\}\}$?

2. How would one formally define an ordered triple? An ordered quadruple?

Problem 42. Show that the composition of one-to-one functions is one-to-one by letting $g : \mathbf{B} \rightarrow \mathbf{C}$ and $h : \mathbf{A} \rightarrow \mathbf{B}$ be one-to-one functions and showing that $g \circ h : \mathbf{A} \rightarrow \mathbf{C}$ is one-to-one.

Problem 43. Show that the composition of onto functions is onto.

The word *identity* is used in a number of different contexts in mathematics. The function $f : \mathbb{R} \rightarrow \mathbb{R}$ defined by $f(x) = x$ is called the *identity* function because it identifies every number with itself. Similarly, 0 is called

an *additive identity* because adding 0 to a number does not change the number, so $0 + x$ is identified with x . By the same reasoning, 1 is called the *multiplicative identity*.

Problem 44. Let $\mathbf{S}$ denote the set of all bijections from $\mathbf{A}$ to itself. According to Problems 42 and 43, $\circ$ is a binary operation on $\mathbf{S}$, that is, $f \circ g \in \mathbf{S}$ whenever f and g are in $\mathbf{S}$. We define a special bijection $i : \mathbf{A} \rightarrow \mathbf{A}$ called the **identity function** on $\mathbf{A}$ as

$$i(x) = x \text{ for all } x \in \mathbf{A}.$$

Show that i is an identity for $\mathbf{S}$, that is,

1. $f \circ i = f = i \circ f$ and
2. $f \circ f^{-1} = i = f^{-1} \circ f$

for all $f \in \mathbf{S}$. This shows that i serves as an identity for composition in the same way that 0, 1, $\emptyset$ work as identities for $+$, $\times$ and $\oplus$, respectively.

Problem 45. Consider the list below of all 6 different bijections from $\{0, 1, 2\}$ onto itself. For example p maps $0 \rightarrow 1$, $1 \rightarrow 2$ and $2 \rightarrow 0$. Make a composition table for these 6 bijections, illustrating Problems 42, 43 and 44 above.

i	p	q	f	g	h
012	012	012	012	012	012
012	120	201	021	210	102

tion table for these 6 bijections, illustrating Problems 42, 43 and 44 above.

Problem 46. A function $f : \mathbf{B} \rightarrow \mathbf{C}$ is said to be **left cancelative** if, for all sets $\mathbf{A}$ and all functions $g, h : \mathbf{A} \rightarrow \mathbf{B}$,

$$f \circ g = f \circ h \text{ implies } g = h.$$

Show that f is left cancelative if and only if f is one-to-one.

Problem 47. A function $f : \mathbf{B} \rightarrow \mathbf{C}$ is said to be **right cancelative** if, for all sets $\mathbf{D}$ and all functions $g, h : \mathbf{C} \rightarrow \mathbf{D}$,

$$g \circ f = h \circ f, \text{ implies } g = h.$$

Show that f is right cancelative if and only if f is onto $\mathbf{C}$.

Chapter 3

Counting

A more formal title for this chapter would be *combinatorics*. Many problems involving probability and statistics require knowing how many elements are in a particular set. Consider poker hands. What is the probability that a hand of five cards has two aces? To answer this, we would divide the number of five-card hands having two aces by the total number of five-card hands. We could list all possible five-card hands and then count the number of these hands that have two aces. Doing this would give us real insight into the problem, so listing is a very good way to solve counting problems. Many times after you start listing elements, you'll find ways to count the elements without listing all of them. If we could count the number of five-card hands having two aces without listing *all* of them, that would be more efficient. In this chapter, to solve a problem will mean to convince the class that you have counted all the items correctly. Perhaps you'll list all the items, or perhaps you'll list a smaller example to demonstrate the counting technique you used.

Problem 48. *The standard license plate for a non-commercial vehicle titled in the State of Maryland consists of six characters. The first character must be one of the nine digits 1, 2, ..., 9. Each of the next three characters must be a letter of the alphabet other than i, o, q, or u. Each of the last two characters must be one of the ten digits 0, 1, 2, ..., 9. How many standard license plates for non-commercial vehicles can be issued by the State of Maryland?*

Problem 49. *There are six area codes used for Maryland telephone numbers: 227, 240, 301, 410, 443, 667. Following each area code are a three-digit "prefix" and a four-digit "exchange." The prefix may not be the number 555, or begin with the number 0. How many telephone numbers can be issued for the State of Maryland?*

Problem 50. *Andrew, Bob, Carly, and Diane are the only entrants in a prize giveaway. Both prizes are the same model of PlayStation. In how many ways can two winners be chosen from them?*

Problem 51. *Andrew, Bob, Carly, and Diane are the only entrants in a prize giveaway. The first prize is an Audi TT, and the second is a Ford Focus. No one person is allowed to win both prizes. In how many ways can the prizes be awarded?*

Problem 52. *Andrew, Bob, Carly, and Diane are the only entrants in a prize giveaway. The first prize is an Audi TT, and the second is a Ford Focus. It is allowable for the same person to win both prizes. In how many ways can the prizes be awarded?*

Definition 16. *A set M is **finite** if there is a nonnegative integer n so that M has n elements and does not have $n + 1$ elements. If the set M has n elements but does not have $n + 1$ elements, then we write $|M| = n$ and say that M is an n -**element set**. A set is **infinite** if it is not finite.*

The next two theorems simply give names to the tools you probably used to solve the last few problems.

Theorem 1. Multiplication Principle. *If each of m and n is a positive integer, A is an m -element set, and B is an n -element set, then $|A \times B| = mn$ or restated, $|A \times B| = |A||B|$. This is sometimes called the Fundamental Principle of Counting.*

Theorem 2. Generalized Multiplication Principle. *Suppose that n is a positive integer and each of $A_1, A_2, \dots, A_n$ is a finite set. Then $|A_1 \times A_2 \times A_3 \times \dots \times A_n| = |A_1||A_2| \dots |A_n|$. This is sometimes called the Generalized or Fundamental Principle of Counting.*

Example 7. *Suppose a mathematician has four different pairs of pants, three different shirts, and five different hats. How many outfits can s/he make assuming an outfit consists of exactly one pair of pants, exactly one shirt and exactly one hat?*

Problem 53. *There are thirteen cards of each of the four suits in a fifty-two card deck. How many possible four-card hands are there containing a card from each of the four suits?*

Problem 54. *Sherwoodn't Cars sells five models of car in three colors. How many different cars could you see in Sherwoodn't's lot, ignoring accessories and options?*

Problem 55. *A program to produce greeting cards has 100 pictures to choose from and twenty-five sayings. How many different cards can the program produce?*

Problem 56. *Suppose there are four 400-level mathematics courses: MATH 406, 402, 465, and 482, offered in a particular semester, and you want to take two of them. How many options do you have?*

Problem 57. Consider the algorithm below.

```

Let i = 1
While i < 7 do
  Let j=2
  While j < 6 do
    Print ''Here is an ordered pair'', (i,j)
    j = j + 1
  End j While Loop
  i = i + 1
End i While Loop

```

How many ordered pairs would this program print?

Problem 58. Suppose that a web site has you choose a username and a password. The username must consist of ten alphanumeric characters. The password must consist of seven alphanumeric characters, the last of which must be numeric and the first of which must be alphabetical.

1. How many usernames are possible if they are not case-sensitive?
2. How many passwords are possible if they are not case-sensitive?
3. How many usernames are possible if they are case-sensitive?
4. How many passwords are possible if they are case-sensitive?

In counting the number of ways we can select several objects from a particular finite set, two questions arise: Does order matter? Is repetition (replacement) allowed?

Definition 17. Let A be a set. By an **unordered sample** of size n chosen from A we mean an n -element subset of A . By an **ordered sample** of size n chosen from A we mean an n -element sequence of (not necessarily distinct) elements of A . In this context, the set A is called the **population** from which the samples are drawn.

For example, $\{1, 2, 3\}$ and $\{2, 3, 1\}$ are the same unordered sample of size 3 from the population of positive integers since order doesn't matter in the specification of a subset. In contrast, $(1, 1, 3)$ and $(1, 3, 1)$ are different ordered samples of size 3 from the same population.

Using this language, we can restate Problem 50 as “How many unordered samples of size two can be chosen without replacement (or “without repetition”) from the four-element “population” (Andrew, Bob, Carly, Diane)?” In Problem 51, we sought an ordered sample of size two without replacement (repetition) from the same population. Finally, in Problem 52 we desired an ordered sample of size two “with replacement” (or “with repetition”).

Example 8. Suppose you are working on a ten-digit keypad to open a door. You know the combination is exactly three digits long.

1. How many choices are there if you may use a number multiple times and the order matters?
2. How many choices are there if you may not use a number multiple times and the order matters?
3. How many choices are there if you may not use a number multiple times and the order does not matter?
4. How many choices are there if you may use a number multiple times and the order does not matter?

Ordered Samples with Repetition Allowed (n-tuples)

The expression $(3, -5)$ is an example of an ordered pair; $(-1, 0, \frac{1}{2})$ is an ordered triple; and $(7, \frac{1}{4}, \pi, z)$ is an ordered quadruple. If n is a positive integer and A is a set and $a_i \in A$ for each integer $i \in \{1, \dots, n\}$, then $(a_1, a_2, \dots, a_n)$ is an ordered n -tuple.

Theorem 3. If each of n and k is a positive integer and A is an n -element set, then the number of ordered k -tuples that can be selected from A is n^k .

Theorem 4 is simply a restatement of Theorem 3 using fancy words!

Theorem 4. If each of n and k is a positive integer and P is an n -element set (population), then the number of ordered samples of size k that can be drawn with replacement (repetition) from P is n^k .

Ordered Samples without Repetition (permutations)

Theorem 5. If n and k are positive integers with $k \leq n$ and P is an n -element set (population), then the number of ordered samples of size k that can be drawn without replacement from P , is $(n-0)(n-1)(n-2) \cdots (n-(k-1)) = n(n-1)(n-2) \cdots (n-k+1)$.

Definition 18. If n is a non-negative integer then **n factorial** is denoted by $n!$ and defined by

$$n! = \begin{cases} 1 & \text{if } n = 0 \text{ or } n = 1 \\ n \cdot (n-1) \cdots 2 \cdot 1 & \text{if } n > 1 \end{cases} \quad (\text{the product of integers 1 to } n)$$

Problem 59. Let each of n and k represent positive integers with $k \leq n$ and show that $n(n-1)(n-2) \cdots (n-k+1) = n!/(n-k)!$.

The next theorem is merely a restatement of Theorem 5 when $k = n$.

Theorem 6. If P is an n -element population, then the number of ordered samples of size n that can be drawn without replacement from P is $n!$.

Definition 19. If n is a positive integer and S is an n -element set, then a *permutation of S* is a bijection on S .

Example 9. Let $S = \{1, 2, 3\}$. One permutation of S would be the bijection $f : S \rightarrow S$ defined by $f(1) = 2, f(2) = 3$, and $f(3) = 1$. Typically, we omit all the function notation and just write the range of f as $(2, 3, 1)$. Recall that the order matters when we use $()$ but not when we use $\{\}$.

Problem 60. List all the permutations of $S = \{1, 2, 3\}$.

Problem 61. How many seven-letter strings can be formed using the letters from the word TUESDAY, where no letter may be used twice? How many five-letter strings?

Problem 62. Let $D = \{a, b, c\}$ and $R = \{1, 2, 3, 4, 5\}$. How many functions are there with domain all of D and with range a subset of R ? How many are one-to-one? How many are onto R ?

Problem 63. Let $D = \{a, b, c, d, e\}$ and $R = \{1, 2, 3\}$. How many functions are there with domain all of D and range a subset of R ? How many are one-to-one? How many are onto R ?

Unordered Samples without Repetition (subsets)

Problem 64. Let $D = \{a, b, c, d, e\}$. How many three element subsets are there of D ?

Theorem 7. If n is a positive integer and k is a nonnegative integer not larger than n , then the number of k -element subsets of an n -element set is

$$\frac{n!}{(n-k)!k!}.$$

Problem 65. Compute $\frac{n!}{(n-k)!k!}$ for each of

1. $n = 10, k = 3$,
2. $n = 10, k = 7$,
3. $n = 5, k = 5$, and
4. $n = 5, k = 0$.

Problem 66. Show that

1. if $n = 20$ and $k = 12$ and $j = 8$ then $\frac{n!}{(n-k)!k!} = \frac{n!}{(n-j)!j!}$ and
2. if n is a positive integer and k is a nonnegative integer not larger than n and $j = n - k$ then $\frac{n!}{(n-k)!k!} = \frac{n!}{(n-j)!j!}$.

Problem 67. *Dr. Shannon has a total of six nieces and nephews. She has just won a set of eight different CD's. In how many different ways can she*

1. *Give each child exactly one CD?*
2. *Give away all the CD's so that each child gets at least one without giving any child more than two?*
3. *Give away all the CD's so that each child gets at least one?*

Problem 68. *Suppose that each of m and n is a positive integer and each of A and B is a set such that $|A| = n$ and $|B| = m$. How many functions are there from A to B ? How many are one-to-one? How many are onto B ?*

Problem 69. *Six cards are to be drawn from a standard deck and laid on a table in the order in which they were drawn. How many outcomes are possible in this experiment?*

Problem 70. *You and thirteen of your closest friends have decided to form a club.*

1. *If you decide to elect four officers, a president, a vice president, a secretary, and a treasurer, then how many possible slates of officers are there?*
2. *If you decide that the job of secretary is too much for one person and elect a president, a vice president, a treasurer, and two secretaries, then how many slates are there?*

Problem 71. *To avoid the diplomatic quagmire of deciding who will sit at the head of the table and who at the foot, a group planning peace talks with the single heads of state of four nations decides to seat all four at a round table, where all spots are equally prestigious and powerful.*

1. *How many possible seating arrangements are there?*
2. *How many arrangements are there if you only care who sits next to whom but not on which side of the person each neighbor sits?*

Problem 72. *An elementary-school teacher is directing an after-school parade with twelve of his students. Three of them will be twirling batons, five will be playing cymbals, and four will be doing somersaults. They will be parading in single file.*

1. *How many different parades are possible if he wants to have the twirlers followed by the cymbal players, with the tumblers at the rear?*
2. *How many are possible if he simply keeps together the children who are doing the same thing?*

3. How many would be possible in a free-for-all, where the kids are in any order?
4. How many are possible if the twirlers stay together but the others can be in any order?

Problem 73. A coin is tossed six times and the results, heads or tails on each toss, are recorded in order.

1. How many outcomes are possible?
2. How many of these have exactly one head?
3. How many have at least one head?
4. How many have at least one head and at least one tail?

Definition 20. Suppose that A is a set and $f : A \rightarrow A$ is a function. If $x \in A$ satisfies $f(x) = x$, then we say that x is a **fixed point of A with respect to f** .

Problem 74. Suppose that $A = \{a, b, c, d\}$.

1. How many functions are there on A for which a is a fixed point?
2. How many for which a and b are fixed points?
3. How many for which a , b and c are fixed points?
4. How many functions are there that fix every point of A ?

Problem 75. Suppose $A = \{a, b, c, d\}$. Let $F1$ be the set of functions $f : A \rightarrow A$ for which a is a fixed point. Let $F2$ be the set of functions $f : A \rightarrow A$ for which a and b are fixed points. Let $F3$ be the set of functions $f : A \rightarrow A$ for which a , b and c are fixed points. Let $F4$ be the set of functions for which all four elements are fixed points. What are $|F1 \cap F2|$, $|F3 \cap F2|$, $|F3 \cap F4|$ and $|F1 \cap F2 \cap F3|$?

Problem 76. If six cards are chosen without replacement from a standard deck, how many hands are possible if

1. the six cards can be anything?
2. at least one card is an ace?
3. at least four are clubs?
4. exactly three of the cards are hearts?

Problem 77. The five-member math club decided to hold a raffle, with each member being responsible for selling ten tickets. Each club member bought one ticket and sold nine to non-members. The stubs were then to be thrown into a fish bowl and three winning tickets were to be chosen at random.

1. Of the possible outcomes, in how many would at least one math-club member win?
2. How many outcomes involve no math club member winning?
3. How many outcomes involve exactly two math club members winning?
4. How many outcomes involve all three winners being club members?

Theorem 8. Binomial Theorem. If each of x and y is a real number and n is a non-negative integer, then

$$(x+y)^n = \binom{n}{0}x^n + \binom{n}{1}x^{n-1}y + \binom{n}{2}x^{n-2}y^2 + \cdots + \binom{n}{n-1}xy^{n-1} + \binom{n}{n}y^n.$$

Problem 78. Prove Theorem 8. This theorem may be proved either using tools from this chapter (a counting argument) or using tools from the forthcoming chapter on induction.

Problem 79. 1. Expand $(x+y)^3$ by hand and using the Binomial Theorem.

2. Expand $(x-y)^5$ using the Binomial Theorem.
3. Use the Binomial Theorem to expand $(x-1)^{10}$. When $x=2$, what happens?

Problem 80. What is the coefficient of x^7 in the expansion of $(1+x)^{23}$?

3.1 Project: Unordered Samples and Probability, Inclusion and Exclusion

Unordered Samples and Probability

Problem 81. Suppose that A , B , and C are sets such that $B \subseteq A$, $C \subseteq A$, $B \cap C = \emptyset$, $|A| = 60$, $|B| = 25$, and $|C| = 20$.

1. How many four-element subsets does A have?
2. How many five-element subsets of A contain no elements of B ? How many contain no elements of C ? How many contain no elements of B and no elements of C ?
3. How many six-element subsets of A contain an element of B ? How many contain an element of B or an element of C ?
4. How many seven-element subsets of A contain at least three elements of B ?

Definition 21. Suppose that A and B are sets and $B \subseteq A$. If we randomly choose an element of A , then the **probability** that the element we pick is in B is $|B|/|A|$.

We may now rephrase our last problem in terms of probabilities.

Problem 82. *Suppose that A , B , and C are sets such that $B \subseteq A$, $C \subseteq A$, $B \cap C = \emptyset$, $|A| = 60$, $|B| = 25$, and $|C| = 20$.*

1. *Suppose I choose a five-element set from A . What is the probability that the set contains no elements of B ? What is the probability that the set contains no elements of C ? What is the probability that the set contains no elements of B and no elements of C ?*
2. *Suppose I choose a six-element subset of A . What is the probability that the set contains an element of B ? What is the probability that the set contains an element of B or an element of C ?*
3. *Suppose I choose a seven-element subset of A . What is the probability that it contains at least three elements of B ?*

Consider the simplest game of poker where we start with a fifty-two card deck and draw hands of five cards. The goal is to have the “best” hand. For example, three sevens beats two kings. Similarly three fours and two jacks (a full house) beats three aces (three of a kind). The order, from weakest hand to strongest hand, goes like this: High Card $<$ One Pair $<$ Two Pair $<$ Three of a Kind $<$ Straight $<$ Flush $<$ Full House $<$ Four of a Kind $<$ Straight Flush. We won’t define and discuss all these possibilities. Rather, we will just pick two and ask the relevant question related to probability.

Why should three of a kind beat two pair? Well it must be that you are more likely to find in a five card hand a pair of threes and a pair of kings than three queens. The next two problems will demonstrate this, first by counting and then by probability.

Problem 83. *Suppose we have a fifty-two card deck and a five card hand from that deck.*

1. *How many five card hands are there?*
2. *How many five card hands have two pair in them?*
3. *How many five card hands have three of a kind in them?*

We may now rephrase our last problem in terms of probabilities.

Problem 84. *Suppose we have a fifty-two card deck and draw a five card hand from that deck.*

1. *What is the probability that there are two pair in the hand?*
2. *What is the probability that there are three of a kind in the hand?*

The Principle of Inclusion and Exclusion

Theorem 9. *If A and B are finite sets, then $|A \cup B| = |A| + |B| - |A \cap B|$.*

Problem 85. *Generalize Theorem 9 to three sets. That is, find a nice formula for $|A \cup B \cup C| = \dots$*

Problem 86. *Generalize Theorem 9 to four sets.*

Chapter 4

Induction and Recursion

Induction is a technique that may be used to prove that a given statement is true for every positive integer.

Example 10. *We begin with two statements that we might try to prove are true for every positive integer. Are both of these statements true for every positive integer?*

1. *My car will start on the n^{th} day after yesterday.*
2. *The sum of the first n positive integers is $n(n+1)/2$.*

Suppose I wish to prove that the second statement above is true for every positive integer. Let's check the first few. If $n = 1$ then this says $1 = \frac{1(1+1)}{2}$ which is true. If $n = 2$ then this says that $1 + 2 = \frac{2(2+1)}{2}$ which is true. If $n = 3$ then this says that $1 + 2 + 3 = \frac{3(3+1)}{2}$ which is true. Now, even if we proved this was true for the first 1,000,000 positive integers, we would not know if it was true for $n = 1,000,001$. How do we *prove* that it is true for all positive integers? Let's proceed by contradiction and suppose that it is *not* true for every positive integer. Since it is not true for every positive integer, then there must be a *first* positive integer m for which it is not true. Since it is true for 1, there is a positive integer n such that $m = n + 1$. Since $n < m$, it is true for n , that is,

$$1 + 2 + \cdots + (n-1) + n = \frac{n(n+1)}{2}.$$

Adding $n + 1$ to both sides we have

$$1 + 2 + \cdots + n + (n+1) = \frac{n(n+1)}{2} + (n+1).$$

Using common denominators to combine the terms on the right yields

$$1 + 2 + \cdots + n + (n+1) = \frac{n(n+1) + 2(n+1)}{2}.$$

Simplifying the right hand side leaves us with

$$1 + 2 + \cdots + n + (n + 1) = \frac{(n + 1)(n + 2)}{2}.$$

We assumed that $n + 1$ was the first positive integer for which the statement was false, yet we have just shown that it is true for $n + 1$. We have a contradiction. This means that our assumption that it was not true for all positive integers is false and we may conclude that the statement is true for all positive integers.

Problem 87. Show that each statement is true.

1. The sum of the squares of the first 2 positive integers is $\frac{2(2+1)(2(2)+1)}{6}$.
2. The sum of the squares of the first 3 positive integers is $\frac{3(3+1)(2(3)+1)}{6}$.
3. The sum of the squares of the first n positive integers is $\frac{n(n+1)(2n+1)}{6}$ for all $n \geq 2$.

Problem 88. Show that each statement is true.

1. The sum of the first 2 cubes is $\frac{2^2(2+1)^2}{4}$.
2. The sum of the first 3 cubes is $\frac{3^2(3+1)^2}{4}$.
3. The sum of the first n cubes is $\frac{n^2(n+1)^2}{4}$ for all $n \geq 2$.

Problem 89. A set of numbers is **bounded** if there are two numbers m and M so that for each number x in the set, $m \leq x \leq M$.

1. Show that a set with one element is bounded.
2. Show that a set with two elements is bounded.
3. Show that a set with three elements is bounded.
4. Show that for each positive integer n , a set with n elements is bounded.

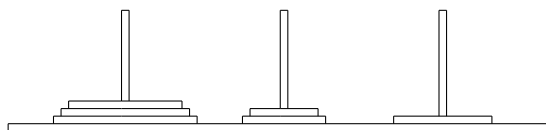
Problem 90. Show that a postage of 12, 13, 14 or 15 cents may be formed using only 4-cent and 5-cent stamps. Then show that every amount of postage of 12 cents or more can be formed using just 4-cent and 5-cent stamps.

Problem 91. Show that the sum of the first n odd positive integers is n^2 .

Problem 92. Assume Theorem 1 (The Multiplication Principle for Counting) is true and prove that Theorem 2 (The Generalized Multiplication Principle for Counting) is true for every positive integer n .

Problem 93. From your Calculus I course, you know that if each of f and g is a differentiable functions, then $(fg)' = f'g + fg'$. Use induction to show that for every positive integer n , if each of $f_1, f_2, f_3, \dots, f_n$ is a differentiable function then

$$(f_1 f_2 f_3 \cdots f_n)' = f_1' f_2 f_3 \cdots f_n + f_1 f_2' f_3 \cdots f_n + f_1 f_2 f_3' \cdots f_n + \cdots + f_1 f_2 f_3 \cdots f_n'.$$



Towers of Hanoi

The Towers of Hanoi is a mathematical puzzle with a history worth Googling. As illustrated above, the puzzle consists of three rods with n disks of different radii which can slide onto any rod. Start with the disks stacked all on one rod, the largest on the bottom, the smallest on the top and in order of size. The object of the puzzle is to move the stack to another rod, obeying the following rules:

1. Only one disk may be moved at a time.
2. Each move consists of taking the upper disk from one of the rods and moving it onto another rod, on top of any other disks that may already be present on that rod.
3. No disk may be placed on top of a smaller disk.

Problem 94. Show that it is possible to solve the Tower of Hanoi puzzle with n discs in $2^n - 1$ moves.

Problem 95. Show that to solve the Tower of Hanoi puzzle on n discs, at least $2^n - 1$ moves must be made.

Problem 96. Show that $1/2 + 1/4 + 1/8 + 1/16 + \dots + 1/2^n = 1 - 1/2^n$.

Problem 97. Show that $3^0 + 3^1 + 3^2 + \cdots + 3^n = (3^{n+1} - 1)/2$ is valid for all non-negative integers, $n = 0, 1, 2, \dots$

Definition 22. A positive integer is a **prime** if it has exactly two positive factors, 1 and itself.

Problem 98. You have an interview with the NSA and they ask you if $p(n) = n^2 - n + 41$ is prime for every positive integer n . Is it?

Problem 99. Show that

$$1 \cdot 2 + 2 \cdot 3 + 3 \cdot 4 + 4 \cdot 5 + \dots + n \cdot (n+1) = n(n+1)(n+2)/3.$$

Problem 100. *Prove that every positive integer is either 1, a prime, or the product of primes.*

Problem 101. *There are many commonly used functions f with domain $\mathbb{Z}^+$ that are not easily defined explicitly. Instead, we tell how to compute f by what we call a **recursive** process. To do this, we*

1. *choose a positive integer b (often $b = 1$),*
2. *give the values of $f(1), f(2), \dots, f(b)$ explicitly, and*
3. *tell how to compute, for any $k > b$, the value of $f(k)$ using the previous values $f(1), f(2), f(3), \dots, f(k-1)$.*

Prove that steps 1-3 can be used to compute $f(n)$ for every positive integer n .

This is exactly what happens when you ask a program to do a recursive call. An advantage of recursively defined functions is that their properties can often be verified by using induction. Your use of quotes in the program and your use of the quotes in the last sentence are incongruous, so I deleted them from the last sentence.

Problem 102. *Consider the following computer program:*

```

n = 1;  a = 2
WHILE  n > 0  DO:
    PRINT  "f(" n ")" = " a
    n = n + 1
    a = 2a - 1
END LOOP
```

Use induction to prove that for each positive integer n the computer will eventually print $f(n) = 2^{n-1} + 1$.

Problem 103. *The **Fibonacci Sequence** is obtained by taking $b = 2$ and defining*

1. $F(1) = 1, F(2) = 1$ and
2. $F(k) = F(k-2) + F(k-1)$ for $k \geq 3$.

Compute $F(3)$ through $F(12)$. Use induction to prove that for all $n \geq 1$,

$$F(1) + F(2) + F(3) + \dots + F(n) + 1 = F(n+2)$$

Problem 104. *Check that $F(3), F(6), F(9)$ and $F(12)$ are even. Then use induction to prove that $F(3n)$ is even for all $n \geq 1$.*

4.1 Project: More Induction

In this chapter we developed and used a method for proving that a given statement is true for all positive integers. This technique is called *strong induction*. Many books treat two techniques of induction, *induction* and *strong induction*, separately. For both induction and strong induction, you first show that the statement is true for $n = 1$. With induction, you assume only that the statement is true for n in order to prove that it is true for $n + 1$. With strong induction, you assume that the statement is true for every number from 1 to n and use this to prove that it is true for $n + 1$. Since both are valid methods, we should state and prove theorems justifying both. In this section we will state an axiom on which these theorems depend, provide guidance on how you might prove them, state the theorems for you to prove, and then provide additional problems using these theorems.

Axiom 4. Well Ordering. *Every non-empty set of positive integers contains a least element.*

Problem 105. *Prove the following two theorems and note where you use the Well Ordering Axiom. The proofs amount to simply repeating again the argument you have used in each of the previous problems that you have done. If you prove this theorem, you will never need to repeat that argument again. You will just quote one of these two theorems!*

Theorem 10. Principle of Strong Induction. *Let $S(n)$ be a statement about an arbitrary positive integer n . If*

1. $S(1)$ is true and
2. $S(k + 1)$ is true whenever $S(m)$ is true for $m = 1, 2, 3, \dots, k$,

then $S(n)$ is true for all positive integers n .

The previous principle is called “strong induction” in order to contrast it with the next principle which is called “induction”. You might prove Theorem 11 directly or you might use Theorem 10 to prove Theorem 11.

Theorem 11. Principle of Mathematical Induction. *Let $S(n)$ be a statement about an arbitrary positive integer n . If*

1. $S(1)$ is true and
2. $S(k + 1)$ is true whenever $S(k)$ is true,

then $S(n)$ is true for all positive integers n .

In each of the induction problems you worked in the main section, you assumed the statement was true for integers $1, 2, 3, \dots, n$ and showed that it

must then be true for $n + 1$. If you review your arguments, you will see that, for most of them, you only used the fact that the statement was true for n in order to prove that it was true for $n + 1$. In those cases, you could have used either induction or strong induction. The exceptions were Problems 90 and 100 where your argument most likely used strong induction even though you can work both using induction. You might also note that there is nothing special about starting with $n = 1$ and for many of the problems, we started with a value for n greater than 1. For each positive integer b we could prove a variant of Theorem 10 ending with

1. $S(b)$ is true and
2. $S(k + 1)$ is true whenever $S(m)$ is true for $b \leq m \leq k$,

then $S(n)$ is true for all positive integers $n \geq b$. Similarly Theorem 11 could be modified to start with b .

Enough theory, let's do some problems. To solve the next problem, we need a recursive definition for factorial.

Definition 23. If n is a non-negative integer then n factorial is denoted by $n!$ and defined by

$$n! = \begin{cases} 1 & \text{if } n = 0 \text{ or } n = 1 \\ (n + 1)! = (n + 1) \cdot n! & \text{if } n > 1 \end{cases}$$

Problem 106. Prove that if $n \geq 5$, then $n!$ is a multiple of 5.

Problem 107. Define a function g recursively by

1. $g(1) = 2$ and
2. $g(k + 1) = 4g(k) - 5$.

Compute $g(7)$. Prove that $g(n) > 2^n$ for all $n \geq 4$.

Recursively defined functions can grow surprisingly fast. As a result, one must be cautious about asking a computer to do recursions since both time and memory are limited. When we were students, if you divided by zero or asked a computer to compute a number that was too large for its memory, it exploded, immediately killing the student. That is why everyone over the age of 30 is smart. ;>

Problem 108. Define a sequence of functions $A_1, A_2, A_3, \dots$ as follows:

1. Let $A_1(n) = 2n$ for all n .
2. For $k = 1, 2, 3, \dots$ let
 - (a) $A_{k+1}(0) = 1$ and

$$(b) A_{k+1}(m+1) = A_k(A_{k+1}(m)).$$

1. Use induction to show that $A_2(n) = 2^n$ for all n .
2. Use induction to show that $A_n(1) = 2$ for all n .
3. Write down $A_3(5)$.
4. Describe $A_3(n)$.
5. The number $A_4(4)$ is so large that I will give an “A” in the course to anyone who can bring me a printout of its value (in normal decimal notation) by the end of the term.

Problem 109. In one of our recent wars, an army captain received orders to deploy his soldiers over a designated enemy area and to instruct them to shoot any resident who came into view.

Now the captain talked with other captains and heard that among the new recruits there were soldiers who for some strange reason didn't buy into this mission. They were even known, in the heat of battle, to shoot their own captains and go AWOL. This worried him, so he devised the following plan.

He would deploy his soldiers, as ordered, but instruct them to situate themselves so that each one was nearest to exactly one other soldier. Each soldier was ordered to watch that nearest soldier to him, and report any suspicious behavior to the captain.

Assuming the number of soldiers was odd, prove that there was at least one soldier who wasn't being watched.

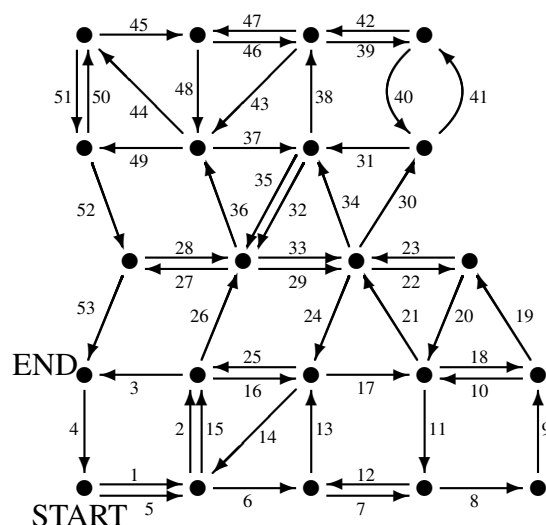
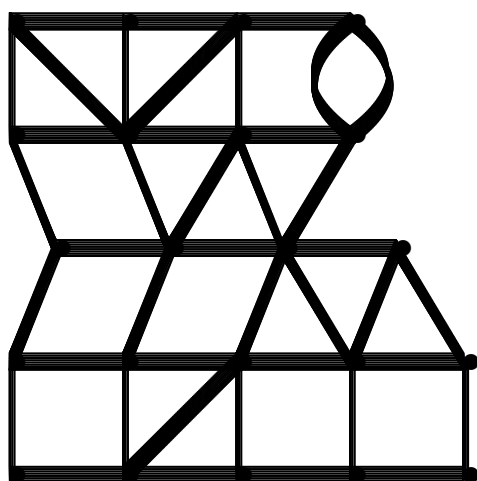
Chapter 5

Euler Paths and Circuits

We begin this chapter with a practical problem. You are offered a job delivering mail in the neighborhood shown below to the left where the lines represent the streets of the neighborhood. The rules are simple:

1. You must deliver mail on each block, so must walk each block.
2. You may start and end anywhere because a friend drops you off and picks you up.

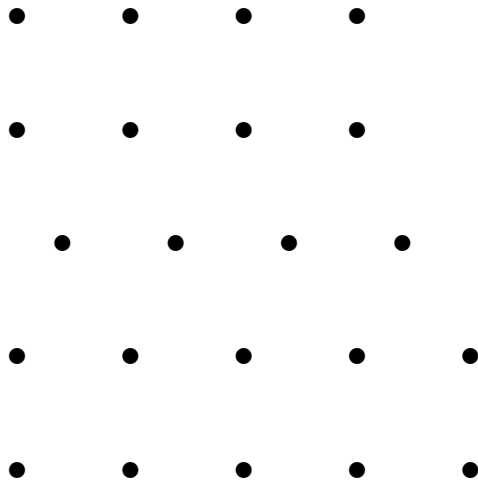
You first note that there are 41 blocks in the neighborhood. Your boss suggests that you use the route he used before he passed it on to you. That route is shown below to the right and consists of 53 blocks. The arrows represent the direction he walked each block and the solid circles represent the intersections. Use the patterns on the next page to find at least one path with fewer blocks than the one your boss suggests. Write out the best path you find in Problem 110.





Problem 110. Write down your best solution to the Mail Carrier Problem, labeling where you start and end, and numbering the blocks as you go.

1. How many blocks are in the neighborhood?
2. How many blocks must you walk in your best solution?

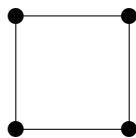


Clearly it is difficult to determine the shortest path, so let's examine some mini-neighborhoods illustrated on the next page.

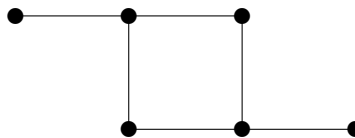
Problem 111. For each graph on the next page write down:

1. the total number of blocks (edges) in the neighborhood, and
2. the smallest number of blocks that you can walk to deliver mail along each block, showing your path.

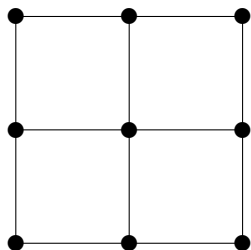
(i)



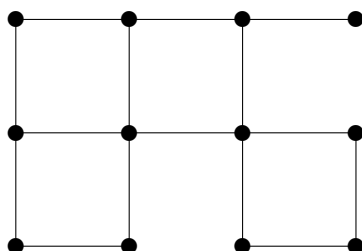
(ii)



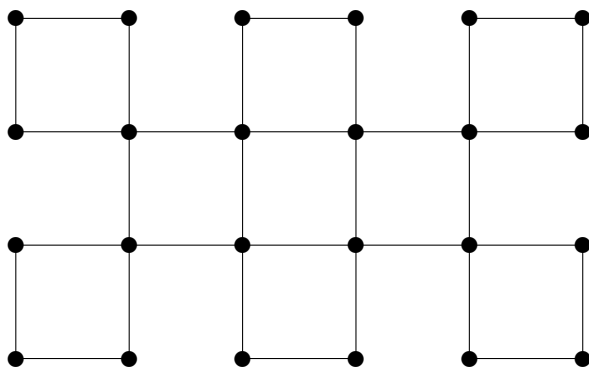
(iii)



(iv)



(v)



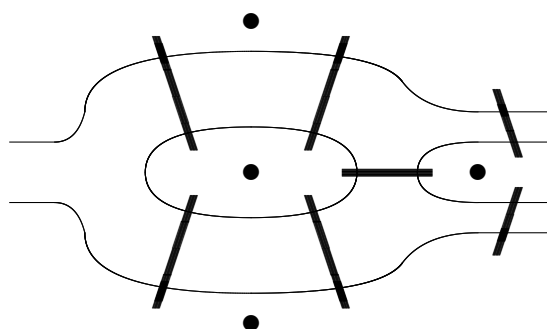


Figure 5.1: Königsberg Bridges

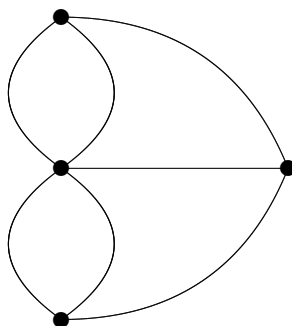


Figure 5.2: Graphical Representation of Königsberg Bridges

The mathematics behind the mail carrier problem dates to eighteenth century east Prussia. The historic city of Königsberg (now Kaliningrad) is broken into four regions by the forking Pregel River. Seven bridges connect the parts of the city as shown in Figure 5.1. Strolling the city, the residents of Königsberg found a challenge. Is there a path that would take them over each bridge exactly once?

Problem 112. Find a path connecting all four regions of Königsberg in Figure 5.1 that takes you over each bridge exactly once.

Suppose we make each region of Königsberg a dot and each bridge a curved line connecting the respective regions. The result is shown in Figure 5.2. Now this problem looks like the mail carrier problem, except that we want to traverse each block *exactly once*.

The mathematical model shown in Figure 5.2 of the Königsberg Bridge Problem in Figure 5.1 is called a *graph* and was invented (discovered?) by Leonard Euler, an eighteenth century resident of Königsberg, in order to solve the Königsberg Bridge Problem.

To solve the mail carrier problem and the Königsberg Bridge Problem, we will first try to decide when it is possible to find a path going over each block or each bridge *exactly once*. These problems may be rephrased mathematically in the language of graphs and Euler paths. Our goal is to determine when a given graph has an Euler path.

Definition 24. A **graph** consists of a finite set of points, called its **vertices**, and a finite set of line segments connecting them, called its **edges**.

Mathematicians do allow for graphs with an infinite number of vertices or edges, but for our class, we will restrict ourselves to the study of graphs with a finite number of vertices and edges.

Definition 25. A **loop** is an edge connecting a vertex to itself. Two or more edges between the same two vertices are called **multiple edges**.

Problem 113. Can you draw a graph which has six vertices, four loops and two multiple edges? If so, do so. If not, why not?

Definition 26. The **degree** of a vertex in a graph is the number of edges coming into it. (A loop counts as two edges.) A vertex is **even** if its degree is an even number, and it is **odd** if its degree is an odd number.

Problem 114. Can you draw a graph with five vertices, each of degree four, which has no loops or multiple edges? If so, do so. If not, why not?

Problem 115. Can you draw a graph with three vertices of degree two and two vertices of degree three? If so, do so. If not, why not?

Problem 116. Can you draw a graph with two vertices of degree two and three vertices of degree three? If so, do so. If not, why not?

Problem 117. Can you draw a graph with one vertex of degree one, two vertices of degree two, three vertices of degree three and four vertices of degree four? If so, do so. If not, why not?

Definition 27. Two vertices are **adjacent** if an edge connects them.

Definition 28. A **path** is a sequence of adjacent vertices.

Definition 29. A **circuit** is a path of at least two vertices which starts and ends at the same vertex without repeating an edge.

Definition 30. An **Euler path** is a path that passes over every edge of the graph exactly once.

Definition 31. An **Euler circuit** is a circuit that passes over every edge of the entire graph.

Problem 118. Can you draw a graph with eight vertices and six edges which contains no circuit at all? If so, do so. If not, why not?

Definition 32. A graph is *connected* if every pair of vertices is connected by some path. A graph is *disconnected* if it is not connected.

Problem 119. Can you draw a connected graph with exactly three edges that does not have an Euler path? If so, do so. If not, why not?

The next theorems tell us which graphs have an Euler path or an Euler circuit and how, if there is one, to find it.

Theorem 12. Odd Vertex Theorem. Suppose A is an odd vertex of a graph.

1. An Euler path that starts at A cannot end at A .
2. An Euler path that does not start at A must end at A .
3. Every Euler path either starts or ends at A .

Theorem 13. Even Vertex Theorem. Suppose B is an even vertex of a graph. Then every Euler path that starts at B must also end at B (and is therefore an Euler circuit).

From these two observations we can establish the following necessary conditions for a graph to have an Euler path or an Euler circuit.

Theorem 14. First Euler Path Theorem. If a graph has an Euler path, then

1. it must be connected and
2. it must have either no odd vertices or exactly two odd vertices.

Theorem 15. First Euler Circuit Theorem. If a graph has an Euler circuit, then

1. it must be connected and
2. it must have no odd vertices.

The two theorems above tell us which graphs do *not* have an Euler path or circuit. We would like to know that all remaining graphs (connected graphs with either zero or two odd vertices) do have Euler paths. To do so requires a few theorems. Try to prove the next three problems, but don't let them slow you down. We can assume them and move on to the rest of the chapter and show them later when you get them.

Problem 120. Show that the sum of an odd number of odd numbers is odd.

Problem 121. Show that the sum of the degrees of the vertices of a graph is even.

Problem 122. Show that every graph has an even number of odd vertices.

Definition 33. *An edge of a connected graph is called a **cut edge** if removal of that edge causes the graph to become disconnected.*

Fleury's Algorithm.

1. *Make a copy of the graph in pencil.*
2. *Start at one of the odd vertices if there are any; otherwise start at any vertex.*
3. *Follow a path in the copy, erasing each edge after you traverse it and numbering the corresponding edge in the original graph. The only rule is to traverse a cut edge only if you are at a degree one vertex of the remaining graph. If you are, then erase that vertex, traverse the cut edge, and then erase the cut edge.*
4. *Stop when there is no edge you can traverse.*

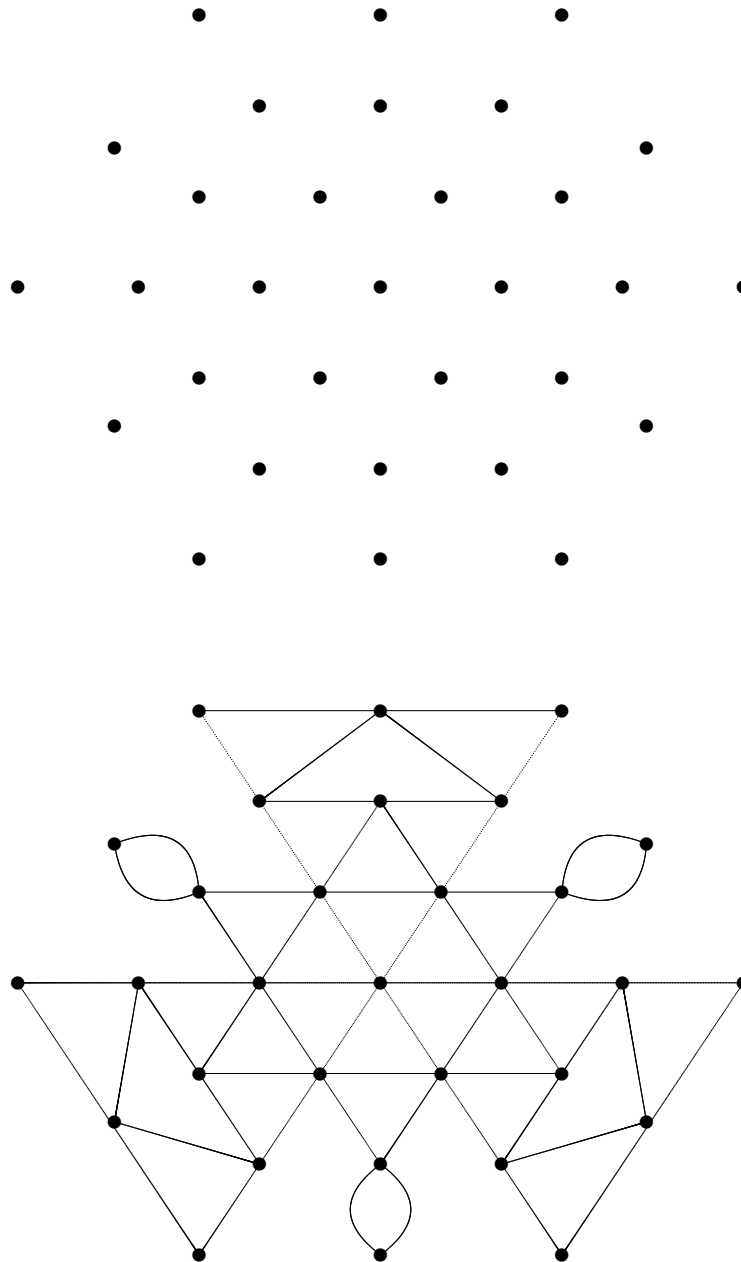
Theorem 16. Fleury's Theorem. *If a graph is connected and has either no odd vertices or exactly two odd vertices, then Fleury's Algorithm will produce an Euler Path.*

See the Project section for a proof of Fleury's Theorem. There we will show that, when we apply Step 4, we will have an Euler Path.

Theorem 17. Second Euler Path Theorem. *If a graph is connected and has exactly 2 odd vertices, then it has an Euler path.*

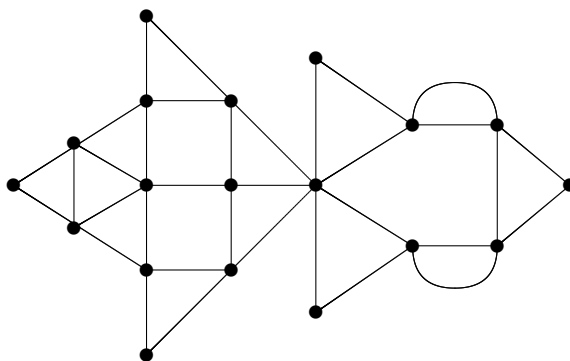
Theorem 18. Second Euler Circuit Theorem. *If a graph is connected and has no odd vertices, then it has an Euler circuit (which is also an Euler path).*

Example 11. Use the previous two theorems to determine if there is an Euler Path or circuit in the graph below. If there is, use Fleury's Algorithm to find one.

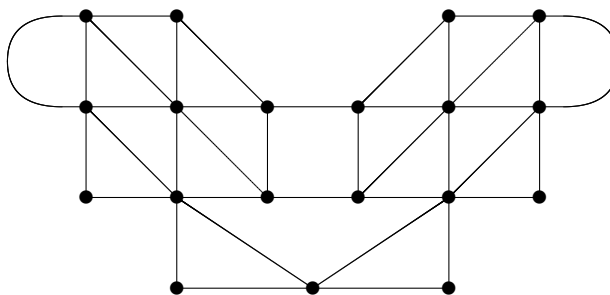


Problem 123. For each graph below, determine if it has an Euler path or circuit. Defend your conclusions in complete sentences. For those that have an Euler path or circuit, apply Fleury's Algorithm to trace one on the graph by marking the start and end, and numbering the edges.

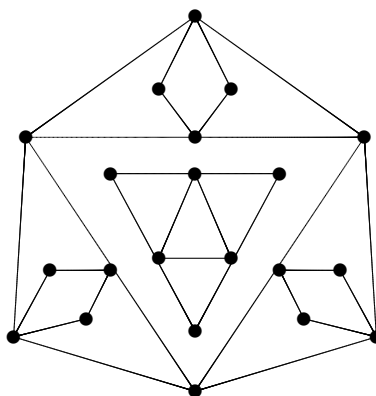
Graph 1



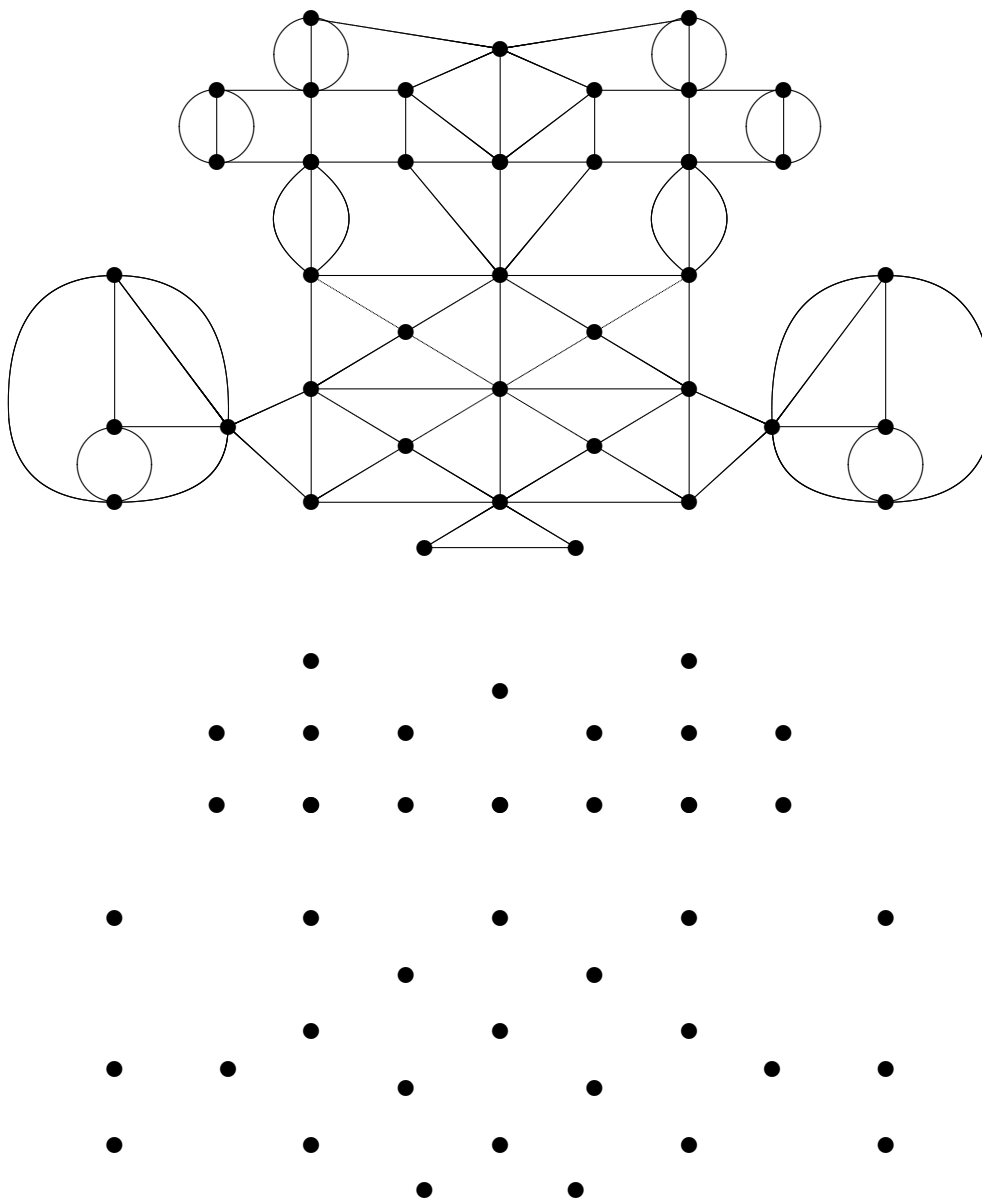
Graph 2



Graph 3



Problem 124. Use Fleury's Algorithm to find an Euler path for the graph below.



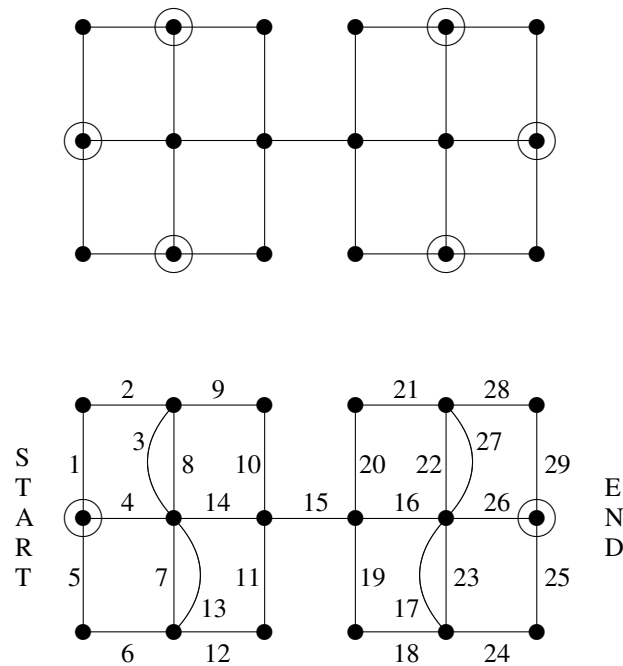


Figure 5.3: Eulerization of the Mail Carrier Problem

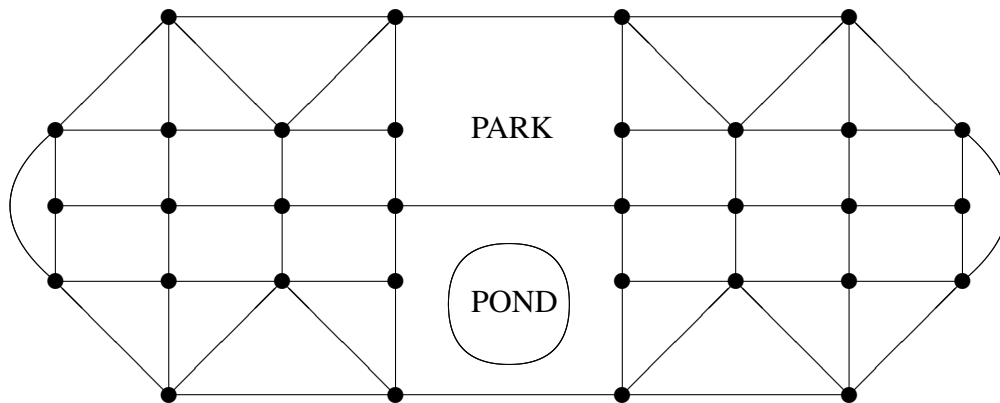
Returning to our mail carrier problem, consider the problem of delivering mail in the top neighborhood illustrated in Figure 5.3. Because it has six odd vertices, we know that the graph has no Euler path and thus we can't find an Euler Circuit. Therefore, we can't travel down each block exactly once. What is the smallest number of blocks that we can walk and still deliver all the mail? To answer this we apply a process called the **Eulerization** of the graph. In the bottom copy of the graph in Figure 5.3, we have inserted four new edges so that the resulting graph has exactly two odd vertices. This is the minimum number of edges that will reduce the number of odd vertices to two. Because we can't build new roads (we are mail carriers, not city planners), the added edges represent the streets we must walk over twice, not new streets. The new graph is connected with exactly two odd vertices, so has an Euler Path of length $25(\text{original edges}) + 4(\text{added edges}) = 29(\text{total edges})$ blocks. This is the shortest possible path that will cover each block in the neighborhood.

Returning to our mail carrier problem, consider the problem of delivering mail in the top neighborhood illustrated in Figure 5.3. Because it has six odd vertices, we know that the graph has no Euler path and thus we can't find an Euler Circuit. Therefore, we can't travel down each block exactly once. What is the smallest number of blocks that we can walk and still deliver all the mail? To answer this we apply a process called

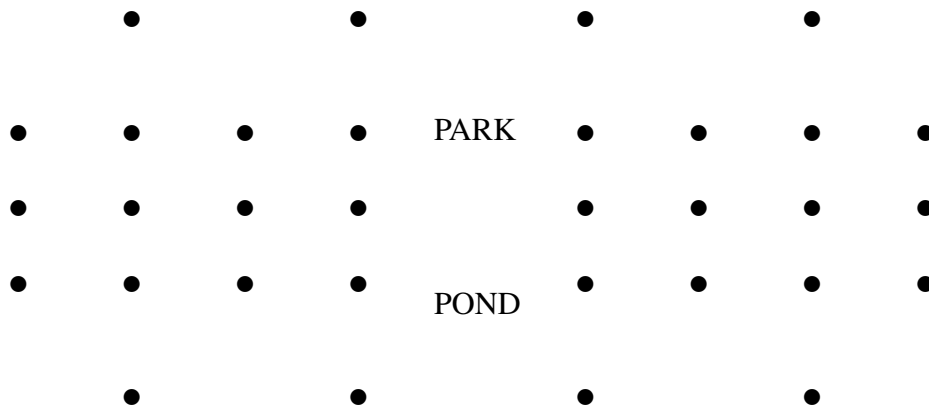
the **Eulerization** of the graph. In the bottom copy of the graph in Figure 5.3, we have inserted four new edges so that the resulting graph has exactly two odd vertices. This is the minimum number of edges that will reduce the number of odd vertices to two. Because we can't build new roads (we are mail carriers, not city planners), the added edges represent the streets we must walk over twice, not new streets. The new graph is connected with exactly two odd vertices, so has an Euler Path of length $25(\text{original edges}) + 4(\text{added edges}) = 29(\text{total edges})$ blocks. This is the shortest possible path that will cover each block in the neighborhood.

Problem 125. Apply the Eulerization process to the original Mail Carrier Problem in Problem 110 at the beginning of the chapter. What is the minimum number of blocks that must be walked in order to deliver all the mail?

Problem 126. You are transferred to a new neighborhood, shown below. Apply the Eulerization process to this new neighborhood so that you add the minimum possible number of edges.



Problem 127. Using the dots below, copy your Eulerized graph above lightly with a pencil. Use Fleury's algorithm to find an Euler path for this graph. Be sure to mark the start and the end.



5.1 Project: Proof of Fleury's Theorem

As we will see in the next chapter, optimization problems often do not have a known and efficient solution that always works. Fleury's Theorem says that the Mail Carrier Problem is one of the exceptional ones that does.

Notice that Fleury's Algorithm can be applied to any graph whereas Fleury's Theorem applies only to special graphs. Assume we apply Fleury's Algorithm to some graph starting at some vertex A . Each time we traverse an edge we erase it, thereby ensuring that we will not traverse it more than once. Since the number of edges is finite, we must eventually stop at Step 4 because there is no edge we can traverse. Now, there are only three possible reasons why there would be no edge we could traverse.

- R1. We are at a degree 0 vertex, having erased all other vertices and all edges. In that case we have traversed all edges (exactly once) and have therefore found an Euler Path! :)
- R2. We are at a degree 0 vertex but there are remaining edges we have not traversed. In that case we can not extend our path to an Euler Path. :(
- R3. We are at a vertex of degree greater than one, but every edge connecting to it is a cut edge. In that case Step 2 prevents us from traversing any of these edges to complete an Euler Path. :(

We will prove Fleury's Theorem by showing that, under the conditions it imposes, neither R2 nor R3 will happen.

Definition 34. A *subgraph* of a graph is a subset of the vertices of the graph along with the edges that connected them in the original graph.

Let $\mathbf{G}$ be a connected graph and let e be a cut edge of $\mathbf{G}$ connecting vertex A with vertex B . Let $\mathbf{X}$ be the subgraph of $\mathbf{G}$ made up of all vertices

C such that there is a path from C to A that does not go over e . In particular, A is in $\mathbf{X}$. Let $\mathbf{Y}$ be the subgraph made up of all of the remaining vertices.

Problem 128. *Show that a vertex D of $\mathbf{G}$ is in $\mathbf{Y}$ if and only if there is a path from D to B that does not go over e . In particular, B is in $\mathbf{Y}$.*

Problem 129. *If e is a cut edge, the subgraphs $\mathbf{X}$ and $\mathbf{Y}$ are both connected.*

Problem 130. *The subgraphs $\mathbf{X}$ and $\mathbf{Y}$ each contain an odd vertex.*

Problem 131. *Assume Fleury's algorithm is applied to a connected graph. Then, for each non-negative integer n , the graph formed by the vertices and edges remaining after traversing n edges is connected.*

Problem 132. *Show that, if Fleury's Algorithm is applied to a connected graph, then R2 can not happen.*

Up to now we have not mentioned the conditions Fleury's Theorem imposes on the number of odd vertices: there are either no odd vertices or exactly two odd vertices. We will need this hypothesis to show R3 can not happen either.

Problem 133. *Assume Fleury's Algorithm is applied to a connected graph with two odd vertices. Let P be the odd vertex where the path begins and let Q be the other odd vertex. Prove the following is true for each non-negative integer n . If the path traverses n edges, then either*

1. *we are at vertex Q and there are no odd vertices in the remaining graph or*
2. *we are not at vertex Q and there are exactly two odd vertices in the remaining graph: Q and the vertex we are at.*

Problem 134. *Assume Fleury's Algorithm is applied to a connected graph with no odd vertices. Let Q be the vertex where the path begins. Prove the following is true for each non-negative integer n . If the path traverses n edges, then either*

1. *we are at vertex Q and there are no odd vertices in the remaining graph or*
2. *we are not at vertex Q and there are exactly two odd vertices in the remaining graph: Q and the vertex we are at.*

Problem 135. *Use the facts you have established to explain why R3 can not happen when Fleury's Algorithm is applied to a connected graph with either no odd vertices or exactly two odd vertices. It follows that the path must end at Step 4 for reason R1, and therefore must be an Euler Path.*

Chapter 6

Traveling Salesman Problems

Phones have the capability to allow you to input multiple locations and compute a path that visits all locations while minimizing your choice of either time or distance. The unit most likely uses one of the algorithms in this chapter. The Traveling Salesman Problem (TSP) models a variety of different real world problems where we seek to minimize the time required to do something:

work orders, where vertices represent repair jobs and weights represent times required to re-tool for the next job;

jobs on a machine, where vertices represent tasks and weights represent times to reconfigure the machine for the next task;

circuit boards, where vertices represent holes to be drilled in the board and weights represent times to rotate the board into the new position.

Returning to your employment status... Having determined that the shortest mail route required nine-hour days, you've decided to look for a job as a traveling salesman. A health food supplier is looking for someone to make deliveries to five stores in the region. Figure 6.1 shows the five stores and the distances between them in miles. You will be paid for each trip that visits all five stores.

This looks good since you can travel at your own pace and won't have to worry about graph theory! Since you live near *A*, you decide on the loop,

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$$

which is

$$60 + 70 + 65 + 100 + 90 = 385 \text{ miles.}$$

On second thought, you observe that you *could* travel the loop,

$$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow A$$

which is only

$$60 + 40 + 110 + 55 + 100 = 365 \text{ miles.}$$

Your brain starts to hurt when you wonder, “What is the shortest route?” and “Just how many routes are there?” Back to graph theory and counting...

Problem 136. *List all the possible routes starting and ending at A, and hitting every other store exactly once. List the total distance traveled for each route. What is a (the?) shortest route? What is a (the?) longest route? Could there be a shorter route starting and ending somewhere else?*

Let’s rephrase this problem in the language of graph theory.

Definition 35. A **weighted graph** is a graph in which each edge has been assigned a positive number called its **weight**.

Definition 36. The **weight** of a circuit is the sum of the weights of its edges.

Definition 37. A circuit which passes through every vertex exactly once is called a **Hamilton circuit**.

Definition 38. A **minimum weight Hamilton circuit** is a Hamilton circuit that has the smallest possible weight of all Hamilton circuits.

In graph theory terms, the **TSP** is the problem of finding a minimum weight Hamilton circuit. Notice that there is an edge between every pair of stores in Figure 6.1. Such a graph is called *complete*.

Definition 39. A **complete graph** is a graph in which every pair of distinct vertices is connected by an edge.

Problem 137. *How many Hamilton circuits are there in a complete graph on three vertices? On four vertices? On twelve vertices? On N vertices?*

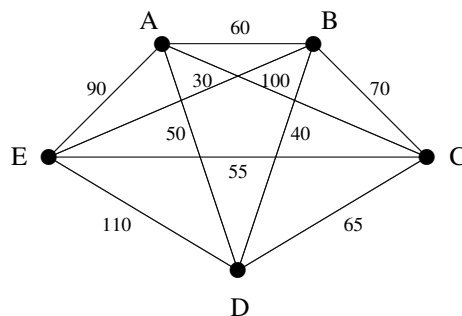


Figure 6.1: Health Food Stores

The TSP is more complex for non-complete graphs. As we have seen, a complete graph has many Hamilton circuits. In contrast, a non-complete graph may have no Hamilton circuit, and the problem of determining if a non-complete graph has a Hamilton circuit at all is a difficult one. Therefore we will consider complete graphs in this chapter except for the last problem of the chapter before the project section. We will study three different algorithms for solving the TSP, each of which is not completely satisfactory for a different reason. The first method we study is the one you applied above.

Brute Force TSP Algorithm.

1. *List all Hamilton circuits.*
2. *Compute the total weight of each circuit in the list.*
3. *Pick out a minimum weight circuit.*

This algorithm finds all optimal solutions, since we check every possible solution. However, the time required to execute the algorithm grows rapidly as we add new vertices. Suppose we have a weighted graph with 12 vertices. To decide if this graph has an Euler path, and then to use Fleury's Algorithm to find that Euler Path, might take 10 minutes. The time required to solve the TSP with the Brute Force Algorithm for a similar sized graph is somewhat longer.

Problem 138. *Suppose you want to use the Brute Force Algorithm to solve the TSP for a graph with 12 vertices, that you can compute the weight of one Hamilton circuit in 10 minutes and that the fate of the world rests on your results. How long will it take you to check all of the different circuits?*

This problem suggests that we consider other more practical algorithms. The next two algorithms are called **greedy algorithms** because at each step we do whatever is most advantageous to us at that moment, without planning ahead.

Nearest Neighbor Greedy Algorithm.

1. *Choose any vertex as a starting point.*
2. *At each step, go to any one of the closest remaining vertices.*
3. *Once every vertex has been visited, return home.*

Now, repeat this process starting at another vertex. Once you have created circuits using each vertex as a starting point, compare the weights of all the circuits and choose a circuit of minimum weight.

Problem 139. Apply the Nearest Neighbor Greedy Algorithm to the Health Food Store problem in Figure 6.1. You need to start by drawing 5 copies of the vertices of the graph, one for each starting place. Circle the starting vertex of each one, and then insert the edges that you use for your circuit numbering them as you add them. Write under each one the weight of the path you get. Finally, tell which path is the shortest. That is your solution to the TSP.

The Nearest Neighbor Greedy Algorithm is more time efficient than the Brute Force Algorithm. Our next algorithm, The Cheapest Link Greedy Algorithm, is even more time efficient.

Cheapest Link Greedy Algorithm.

1. Make a copy of the vertices of the original graph.
2. Repeat Step 3 until you have a Hamilton circuit.
3. Add a minimal weight edge from your original graph to your copy so that
 - (a) if added to your copy, it will not create a degree three vertex, and,
 - (b) if added to your copy, it will not close a circuit unless that circuit is a Hamilton circuit.

Problem 140. Apply the Cheapest Link Greedy Algorithm to Figure 6.1. Draw the 5 vertices, then add the edges one at a time, numbering them as you go, until you have a circuit. What is the weight of this circuit?

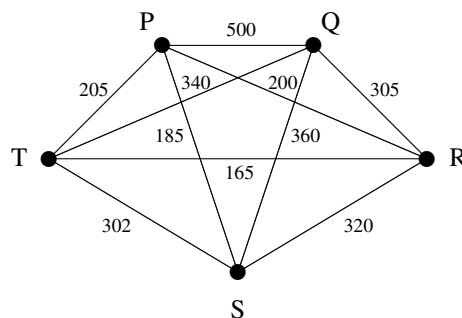


Figure 6.2: Weighted Graph

Problem 141. Use the Brute Force Algorithm to solve the TSP for Figure 6.2.

Problem 142. Use the Nearest Neighbor Greedy Algorithm to solve the TSP for Figure 6.2.

Problem 143. Use the Cheapest Link Greedy Algorithm to solve the TSP for Figure 6.2.

We have seen the NNGA give us minimal weight Hamilton circuits while the time efficiency of the CLGA can come at the cost of a slightly less than minimal weight circuit.

Problem 144. In the previous two problems, the NNGA gives the minimal weight circuit found by the BFA and the CLGA gives the second to minimal weight circuit. Can you find an example in which the CLGA gives a minimal weight circuit but the NNGA does not?

Does the NNGA, like Fleury's Algorithm, always give us an optimal solution? The answer is "No".

Problem 145. Find an example of a weighted graph with four vertices in which neither of the Greedy Algorithms produce a correct answer to the TSP no matter which allowable choices are made. (This proves mathematically that greed does not always pay!)

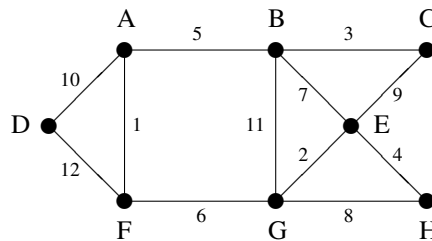


Figure 6.3: An Incomplete Weighted Graph

Problem 146. The weighed graph in Figure 6.3 is not complete.

1. Without using any of our algorithms, find a minimum weight Hamilton circuit.
2. Try to apply the greedy algorithms to this problem. What happens?

Since 1971, when it first gained public attention, considerable effort has gone into finding a good solution to the Traveling Salesman Problem, but no solution has been found that is quick, efficient, easy to apply (like Greedy algorithms) and guaranteed to work (like the Brute Force Algorithm). Considerable evidence suggests that no such solution exists at all.

6.1 Project: Edges with Equal Weights

The weighted graphs presented in this chapter have all had the property that no two edges have the same weight. If, in the application of the greedy algorithms, there is more than one edge with minimal weight, we simply pick one of them at random. This means that we may get different outcomes depending on the choices we make. See how much variation there is in the outcomes when your class does the following problem.

Problem 147. *Suppose that you need to make a delivery to each of 11 friends and then return home. What's more, your car is in very bad shape and you don't want to drive it any further than absolutely necessary. Call your house A and the houses of your friends B, C, D, E, F, G, H, I, J, K and L. Table 6.1 below indicates the distances between each pair of houses. You need to find a Hamilton circuit through the complete graph on 12 vertices, A, B, C, D, E, F, G, H, I, J, K and L, which has as small a total distance as possible. According to what you learned in Problem 138, you are quite certain that you don't want to use the Brute Force Algorithm to find this circuit.*

	A	B	C	D	E	F	G	H	I	J	K	L
A		8	15	7	6	9	10	16	11	7	9	10
B			9	12	6	9	11	15	12	8	9	6
C				7	10	11	9	5	6	9	10	12
D					8	6	13	10	9	5	11	7
E						10	7	6	9	10	8	9
F							9	8	10	6	5	16
G								7	15	7	6	8
H									5	9	14	10
I										8	19	7
J											12	5
K												6

Table 6.1: table of distances between 12 houses

1. Pick a random Hamilton circuit and compute its total length.
2. Apply the Nearest Neighbor Greedy Algorithm, starting from A (only), to find a Hamilton circuit. What is its total length?
3. Apply the Nearest Neighbor Greedy Algorithm, starting from D (only), to find a Hamilton circuit. What is its total length?
4. Apply the Cheapest Link Greedy Algorithm to find a Hamilton circuit. What is the length of this circuit?

The example in Problem 147 shows how the greedy algorithms are normally used. They are the best methods currently available to efficiently compute

low weight Hamilton circuits when the Brute Force Algorithm is not feasible to use, as illustrated in Problem 138. However, there are instances where they can fail completely.

Problem 148. *Construct a weighted graph with 4 vertices and some edges of equal weight such that both greedy algorithms can be used to obtain either a minimum weight Hamilton circuit or a maximum weight Hamilton circuit, depending on the choices made along the way.*

Chapter 7

Spanning Trees

The left diagram in Figure 7.1 shows seventeen towns and the roads connecting them. The town with the circle has a power plant. Your job as a consultant to the power company is to decide which roads to install power lines on so that every town receives power and you install lines on as few roads as possible. You don't need a power line constructed down every road since you only need to ensure that there is a path for power to travel from the power station to each town.

Problem 149. *Using the diagram at the right of Figure 7.1, sketch a possible way to connect all the towns to the power station. Look for one that places power lines on the minimal number of roads.*

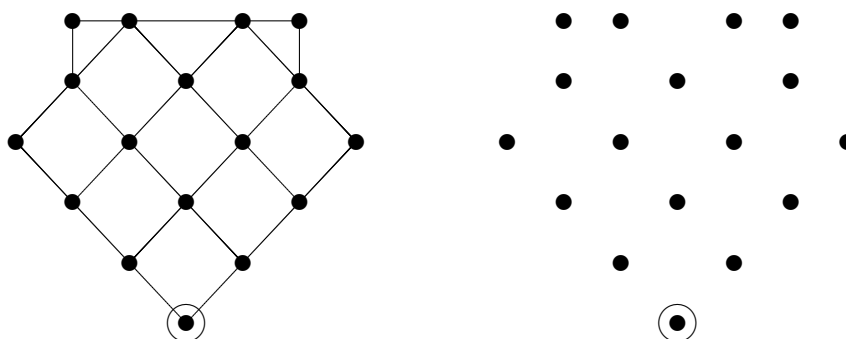


Figure 7.1: Power Grid Spanning Tree

Looking at our solutions, we probably believe they are correct. Still, what would we need to do in order to verify one of them? We would need to check that *no* collection of roads connecting all the towns to the power station has *less* roads than this solution. Because Problem 149 has only 17 towns, we might suspect that this would be easy. Surprisingly, Cayley

studied such problems and proved that there can be up to n^{n-2} different solutions for n towns. If $n = 17$, that is more than 10^{18} , or a billion billion, cases to check! Would you be surprised that the language of graph theory might help?

Definition 40. A *tree* is a connected graph with the property that no graph with the same vertices and a smaller number of edges is connected.

Definition 41. A *spanning tree* of a graph G is a tree whose vertices are the same as the vertices of G and whose edges are among the edges of G .

Problem 150. Is the power grid you created in the last problem a spanning tree?

Here are several equivalent definitions for trees. Interested in a proof? See Section 7.1.

Theorem 19. Equivalent Tree Definitions. For a connected graph G , the following are equivalent.

1. G is a tree.
2. Every edge of G is a cut edge (Definition 33).
3. G has no circuit.
4. The number of edges in G is one less than the number of vertices.

Problem 151. Use Theorem 19 to give three different ways to show why your answer to Problem 149 is correct.

Another real-world application for spanning trees might be to find the best possible collection of roads to plow when the city is snowed in. If we plow out a spanning tree then people may travel from any intersection (vertex) to any other intersection in the entire city by traversing only plowed streets (edges in the spanning tree). For a large city, computing the spanning tree by hand would be impractical. Here is an algorithm that a computer can implement.

Theorem 20. Spanning Tree Algorithm. To construct a spanning tree for a connected graph G , make another copy of the vertices of G . Then

1. add edges from G between these vertices one at a time making sure to never close a circuit, and
2. stop when no additional edge from G can be added without closing a circuit.

Problem 152. Prove that for any connected graph G , the Spanning Tree Algorithm always produces a spanning tree for G .

Applying the Spanning Tree Algorithm to the power grid problem illustrated in Figure 7.1, you could find many different spanning trees to build your power lines. Of course, in the real world, some would cost more to construct depending on both the distance between towns and the terrain between towns. Just as with the shortest path problems, this problem may be modeled mathematically using a **weighted graph** where the weights represent quantities like distances or costs.

Minimum Weight Spanning Tree Problem *Given a weighted graph, find a spanning tree whose total weight is less than or equal to that of any other spanning tree.*

In Figure 7.2, we have added weights to represent either distances (in miles) or road construction costs (in millions of dollars) between our 17 towns.

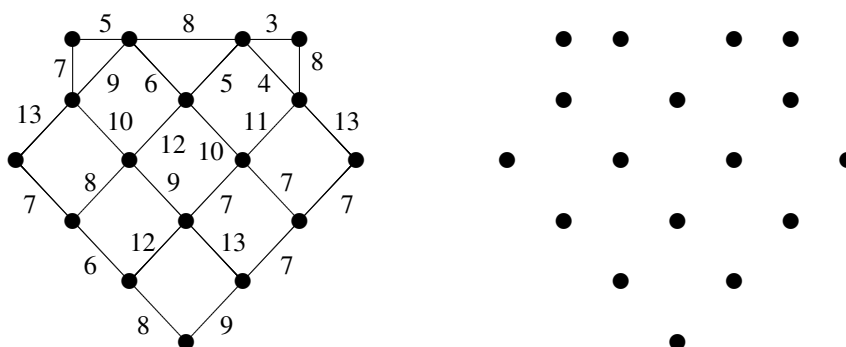


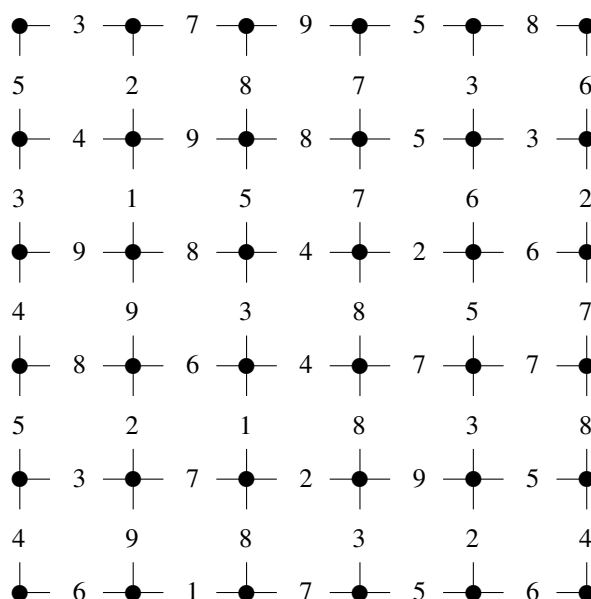
Figure 7.2: Weighted Graph

Problem 153. *Consider the weighted graph in Figure 7.2.*

1. *Using this weighted graph, calculate the total weight of the spanning tree in your solution to Problem 149.*
2. *Is your solution a minimum weight spanning tree?*
3. *Find a spanning tree for the weighted graph in Figure 7.2 that has weight less than your solution to Problem 149.*
4. *What is the least weighted spanning tree that you can find?*

We can now ask if your solution to Part 4 of Problem 153 has minimal total weight among all possible spanning trees. How might we do this without looking at all possible spanning trees? The answer was discovered

by Joseph Kruskal in 1956 and is simply the Spanning Tree Algorithm enhanced by the strategy used in the Cheapest Link Greedy Algorithm. Instead of choosing an arbitrary edge that doesn't complete a circuit, we choose the cheapest (least weight) one. Since Kruskal's Algorithm is also a greedy algorithm, it is practical and efficient to use. But unlike our other greedy algorithms, it *always works*!



A proof is outlined in Section 7.1.

Problem 154. *Apply the Spanning Tree Algorithm to the graph in Figure 7.3 without looking at the weights. What is the total weight of the spanning tree you created?*

Problem 155. *Apply Kruskal's Greedy Algorithm to the graph in Figure 7.3 to find a minimum weight spanning tree. What is the total weight of this spanning tree?*

7.1 Projects: Tree Equivalence and Kruskal's Theorem

Equivalent Tree Definitions. In the next few problems, we investigate how one might discover and prove Theorem 19.

Problem 156. *Draw five different connected graphs in which every edge is a cut edge. Count the number of edges and the number of vertices. Make a conjecture for the relationship between the number of vertices and the number of edges in such a graph.*

Our goal is to find some easily verifiable conditions for a graph to be a tree.

Problem 157. *Show that if an edge of a connected graph is part of a circuit, then that edge is not a cut edge.*

Problem 158. *Show that if a connected graph with more than one vertex has no circuit, then every path either ends in a degree 1 vertex or can be extended to a path that ends in a degree 1 vertex.*

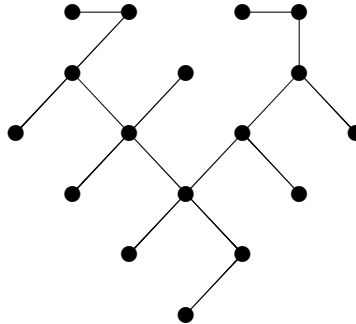
Problem 159. *Let G be a connected graph with n vertices and no circuit. Prove by induction on n that G has exactly $n - 1$ edges.*

Problem 160. *Show that if the number of edges in a connected graph is one less than the number of vertices, then it is a tree.*

Problem 161. *Use the facts you have established to prove the following theorem by showing that line 1 implies line 2, which implies line 3, which implies line 4, which implies line 1.*

Theorem. Equivalent Tree Definitions. For a connected graph G , the following are equivalent.

1. G is a tree.
2. Every edge of G is a cut edge.
3. G has no circuit.
4. The number of edges in G is one less than the number of vertices.

Figure 7.4: Spanning Tree S

Kruskal's Theorem. In the next two problems we will look at Kruskal's Theorem to see why the spanning tree obtained from Kruskal's Algorithm always has the smallest total weight of any spanning tree. In the figure below is an arbitrary spanning tree S for the graph in Figure 7.1. We would like to see why its total weight must be no less than that of the spanning tree obtained from Kruskal's Algorithm.

Problem 162. Apply Kruskal's Algorithm to the graph in Figure 7.2 to obtain a minimum weight spanning tree T . As you do it, number the edges in the order you add them. Find the total weight of T .

Problem 163. What is the total weight of the spanning tree S in Figure 7.4? Make a copy of S in pencil. Transform the spanning tree S into T by adding the edges of T to it, in the order you numbered them, each time removing an edge not in T that closes a circuit. As you do so, fill out a table like this:

Weight of edge added to S from T	Weight of edge not in T removed from S	Amount the weight of S is reduced
...	...	...
...	...	...

When you have finished transforming S into T , add up the reductions you made in the weight of S . Now subtract this amount from the original weight of S to see if you get the weight of T . Write this out *neatly* so that it is clear what you have done. (If you can reduce the weight of S to get the weight of T , then you know that T must have smaller weight than S !)

Problem 164. Using Problem [163](#) as a guide, prove that Kruskal's Algorithm always produces a minimal spanning tree.

Chapter 8

Equivalence Relations and Partitions

Our goal in this chapter is to understand the relationship between partitions and equivalence relations. Before we delve into the formal definitions, let's look at two examples that illustrate our goal.

Suppose we break our class into four subsets by rank: freshmen, sophomores, juniors and seniors. Our collection of subsets is called a partition of the class because the collection has three properties. (1) The union of the subsets is the entire class. (2) The intersection of any two of the subsets is empty. (3) Each subset is non-empty. We can use our partition to define what it means for two students to be equivalent, by saying that two students in the class are equivalent if they have the same class rank. Thus if we have a partition of a set, we have a way to define equivalence.

Now consider the set of all integers. Let's define equivalence for this set by saying that two integers are equivalent if they have the same remainder when divided by 3. For example, 4 and 7 both have remainder 1, so they are equivalent. Since there are only three possible remainders when dividing an integer by 3, we can put all the integers with remainder 0 into one set, put all the integers with remainder 1 into another set and put all the integers with remainder 2 into a third set. These three sets partition the integers because they satisfy the three properties above. (1) Their union is all integers. (2) No number has two different remainders when divided by 3, so the intersection of any two of our sets is empty. (3) Each set is non-empty.

Summarizing, if you have a partition of a set, then you can define what it means for two elements of the set to be equivalent. Conversely, if you have a definition of what it means for elements in a set to be equivalent, then you can create a partition of the set from that definition.

Definition 42. A collection P of sets is said to be **pairwise disjoint** if the intersection of any two sets in the collection is empty.

Definition 43. A collection P of subsets of a set A is said to be a **partition** of the set A if the collection is pairwise disjoint, the union of the sets in P is A and each set is non-empty.

Problem 165. *Construct three different partitions of the integers.*

Let's recall what we know about relations before we move to a special kind of relation called an equivalence relation.

Recall from Definitions 10 and 11 that if **A** and **B** are sets, then the **Cartesian product** of **A** and **B** is the set of all ordered pairs (x, y) where $x \in \mathbf{A}$ and $y \in \mathbf{B}$,

$$\mathbf{A} \times \mathbf{B} = \{ (x, y) \mid x \in \mathbf{A} \text{ and } y \in \mathbf{B} \},$$

and that a **relation** on $A \times B$ is any subset of this set.

Problem 166. *Let $R = \{ (x, y) \mid x \in \{a, b, c, d, e\} \text{ and } y \in \{u, v, w\} \}$. List all the elements of this relation. Is R a function? Create some type of graphical representation for this relation.*

Problem 167. *Let $A = \{1, 2, 3\}$. Write out all elements of $A \times A$. List two examples of relations on the set $A = \{1, 2, 3\}$, one which is not a function and one which is a function.*

Functions are one example of relations and equivalence relations are a second example of relations.

Definition 44. *If R is any relation, then xRy means $(x, y) \in R$.*

Definition 45. *Suppose A is a set and $R \subseteq A \times A$ is a relation on A .*

1. *If for every $x \in A$ we have xRx , then R is said to be **reflexive**.*
2. *If for every $x, y \in A$ satisfying xRy we have yRx , then R is said to be **symmetric**.*
3. *If for every $x, y, z \in A$ satisfying xRy and yRz we have xRz , then R is said to be **transitive**.*
4. *If R is reflexive, symmetric and transitive, then R is said to be an **equivalence relation**.*

Note that for equivalence relations, the domain and range must be the same.

Example 12. *Suppose $X = \{ \text{all people in the world} \}$ and x is related to y means x is a friend of y . Discuss whether the reflexive, symmetric and transitive properties hold. Can we construct a partition based on this relation?*

Example 13. *Suppose $X = \{ \text{students in our class} \}$ and x is related to y means x and y have the same class rank (freshman, sophomore, junior, senior). Discuss whether the reflexive, symmetric and transitive properties hold and foreshadow the equivalence classes. Can we construct a partition based on this relation?*

Problem 168. Let A be the set of all real numbers. Suppose that x is related to y if x and y are equal. In other words, xRy if $x = y$. Which of these properties hold for the relation R ?

1. reflexive (Is x related to x ? I.e. Does $x = x$?)
2. symmetric (If xRy , then is yRx ? I.e. If $x = y$, then does $y = x$?)
3. transitive (If xRy and yRz , then must xRz ?)

Construct a partition of A based on this relation.

Problem 169. Let A be the set of all real numbers. Let xRy if $x \leq y$. Check whether R is reflexive, symmetric and transitive. Is R an equivalence relation? Can we construct a partition based on this relation?

Definition 46. Given two integers x and y we say that x is a **multiple** of y if there is an integer k so that $x = ky$.

Problem 170. Let A be the set of all positive integers. Define a relation R by xRy if x is a multiple of y . Which of these properties hold for the relation R ?

1. reflexive (Is x a multiple of x ?)
2. symmetric (If x is a multiple of y , then must y be a multiple of x ?)
3. transitive (If x is a multiple of y and y is a multiple of z , then must x be a multiple of z ?)

Problem 171. Let A be the set of all integers. Suppose that xRy if x and y have the same remainder when divided by 5. Is R an equivalence relation?

Recall the equivalence relation on our class where two students are related if they have the same class rank. This equivalence relation partitions our class into subsets where everyone in a given subset is related to everyone else in that subset, no person is in two different subsets, and the union of all the subsets is the entire class. The next definition gives us a name for the subsets in the partition.

Definition 47. If R is an equivalence relation on the set A and $a \in A$, then the **equivalence class of a** is the set $\{y \in A \mid yRa\}$ and is denoted by $[a]$.

To determine what the equivalence classes are, just pick an element and ask yourself, “What other elements are related to this element?” Once you’ve done this for a few elements, you’ll understand all the equivalence classes for that particular relation.

Problem 172. For each of the problems 168 through 171 where R turned out to be an equivalence relation, define the equivalence classes.

Up to this point, we were given a relation and we checked to see if it was reflexive, symmetric, and transitive. If it was, then we listed all the equivalence classes and they formed a partition. Thus each equivalence relation yielded a partition. The fact that this always works is stated formally as the following theorem and the proof is the first problem in Section 8.1.

Theorem. If R is an equivalence relation on the set A , then the set of all equivalence classes defined by R form a partition of A .

We now consider the converse of this process, showing that every partition yields an equivalence relation. Suppose we have a set that we partition into a collection of non-empty subsets which are pairwise disjoint and whose union is our set. Then we can define a relation by saying that x and y are related if, and only if, they are in the same subset in the partition. This relation will be an equivalence relation, so we now have a way to create equivalence relations from partitions. In other words, equivalence relations and partitions are the exact same idea written in different mathematical language.

Problem 173. Suppose P is a partition of the set A . Define the relation R on A by xRy if there is $B \in P$ such that x and y are both in B . Prove that R is an equivalence relation on A .

Problem 174. Let $\mathbf{A}$ be the set of points (x, y) in the plane and suppose that $(x, y)R(x', y')$ if (x, y) and (x', y') are the same distance from the origin. Describe the equivalence class for $(0, 2)$, that is, describe $[(0, 2)] = \{(x, y) \in \mathbb{R}^2 \mid (x, y)R(0, 2)\}$. Determine if R is an equivalence relation and if so, describe all the equivalence classes.

Problem 175. Let A be the set of all integers. Suppose xRy if x and y are integers and $x - y$ is a multiple of 3. Determine if R is an equivalence relation and if so, describe the equivalence classes.

Problem 176. Let A be the set of integers. Suppose xRy if x and y are integers and $x + y$ is even. Determine if R is an equivalence relation and if so, describe the equivalence classes.

Problem 177. There is a standard and important notion of equivalence between sets. Let $\mathbf{S}$ denote the collection of all sets. For $X, Y \in \mathbf{S}$, we say that XRY if there is a bijection $f : X \rightarrow Y$ from X to Y . Thus finite sets X and Y are equivalent if and only if they have the same number of elements, but in general they need not be finite. Show that R is reflexive, symmetric and transitive.

8.1 Project: More on Equivalence Classes

Problem 178. Suppose R is an equivalence relation on the set A . Prove that the set of all equivalence classes defined by R form a partition of A .

Problem 179. We might think of two compound sentences in propositional logic to be “equivalent” if they have the same truth tables. For example, the truth table below shows that every implication $p \rightarrow q$ is equivalent to its contrapositive $\neg q \rightarrow \neg p$.

p	q	$p \rightarrow q$	$\neg q \rightarrow \neg p$
0	0	1	1
0	1	1	1
1	0	0	0
1	1	1	1

Table 8.1: 1 is True; 0 is False

Let $\mathbf{A}$ be the (infinite) set of all compound sentences in propositional logic that can be constructed using the sentence variables p and q and the connectives $\neg$, $\wedge$, $\vee$ and $\rightarrow$. For $a, b \in \mathbf{A}$ we say that $a \equiv b$ if a and b have the same truth table.

1. Show that $\equiv$ is an equivalence relation.
2. List of as many non-equivalent compound sentences as you can.
3. How many different equivalence classes are there?

Problem 180. Let $\mathbf{S} = \{a, b, c, d, e, f, g, h\}$, let $\mathbf{T} = \{a, b, c\}$ and let $\mathbf{A}$ be the set of all (2^8) subsets of $\mathbf{S}$. For $X, Y \in \mathbf{A}$, we define XRY if X and Y have the same intersection with $\mathbf{T}$. Determine if R is an equivalence relation and if so, describe the equivalence classes.

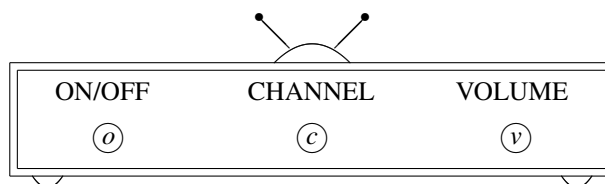
Problem 181. Let $\mathbf{A} = \{0, 1\}^5$ be the set of 32 different five-tuples of 0s and 1s, that is, all sequences (a, b, c, d, e) where $a, b, c, d, e \in \{0, 1\}$. For $x, y \in \mathbf{A}$, define xRy to mean that x and y have the same number of 1s. Show that R is an equivalence relation and write down all the members of each of the different equivalence classes.

Problem 182. Let n is a positive integer and let $\mathbf{A} = \{0, 1\}^n$ be the set of all n -tuples of 0s and 1s. For $x, y \in \mathbf{A}$, again define xRy to mean that x and y have the same number of 1s. How many different equivalence classes does $\mathbf{A}$ have?

Chapter 9

Finite State Machines

Imagine a simple radio with three buttons, an ON/OFF button, a CHANNEL button and a VOLUME button.



The radio has ten channels and five volume levels:

C0, C1, C2, C3, C4, C5, C6, C7, C8, C9, V0, V1, V2, V3, V4.

Pressing button o turns it from OFF to ON or ON to OFF. Regardless of whether the radio is ON or OFF, pressing c advances the channel by one or takes it from C9 back to C0. Regardless of whether it is ON or OFF, pressing v increases the volume by one level or takes it from V4 back to V0.

The radio can be thought of as having $2 \times 10 \times 5 = 100$ different internal states. In state (OFF, C3, V4), pressing o will change it to state (ON, C3, V4). Then pressing the sequence $ccvvv$ will change it from there to channel C5 at volume V2, that is, to state (ON, C5, V2).

Problem 183. *If the radio is in state (OFF, C8, V1) and you press the input sequence $ocvvcovo$, what state will it be in?*

Problem 184. *List three different input sequences that will bring it from state (ON, C7, V3) to state (OFF, C2, V0).*

A radio like this is an example of a Finite State Machine (FSM). In this chapter we will study somewhat more elaborate Finite State Machines: Finite State Transducers and Finite State Acceptors. FSMs are the simplest class of computing machines. They model the logic of machines which require logic circuitry, but are not usually thought of as computers, such as vending machines, washing machines and toasters.

FSMs are characterized by the fact that their action, at any point in their operation, is completely determined by their current state and their current input. They have no long term memory and are therefore unable to carry out any logical process of reviewing the past or anticipating the future.

9.1 Finite State Transducers

Suppose the directions on a vending machine ask the user to:

1. Choose either (P) Peanuts (65¢) or (D) Doritos (45¢).
2. Insert only enough quarters or half dollars to pay for your choice.
3. Press “RETURN”.

In order to execute instructions, the machine must remember, at any time, the user’s selection and the amount of money it has received. Memory will take the form of internal states, one for each possible data set. The machine logic is represented by the directed labeled graph M_1 shown in Figure 9.1. The circles are the internal states that form the vertices of M_1 . The arrows tell us

1. what the output should be for each input and
2. to what new internal state the machine should go.

Unspecified inputs leave the state unchanged and give no output.

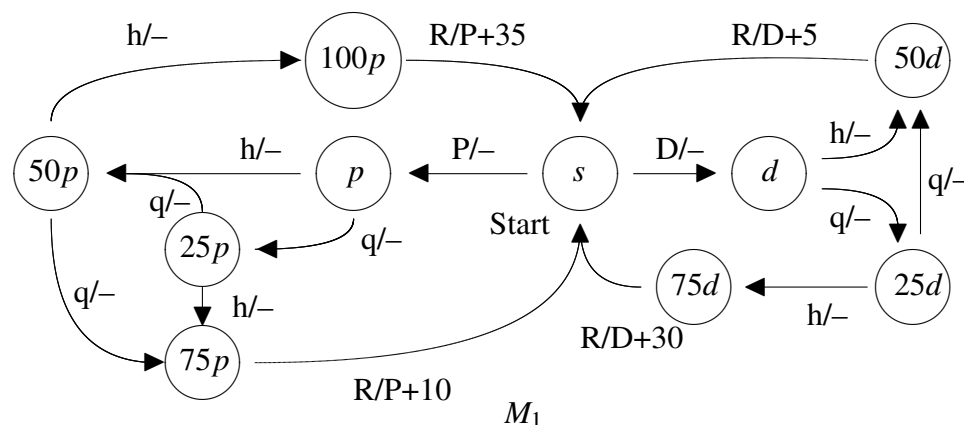


Figure 9.1: Finite State Transducer for the Peanut/Dorito machine M_1 .

The start state is “s”, and each other state has a name that is intended to suggest what it is supposed to remember. The symbol “-” means that there is no output.

Problem 185. Suppose our vending machine in Figure 9.1 is in state “s.” Describe what happens when a user inputs the sequence, “PqghR”. What happens if the user inputs “DhqqqRh”?

In order to describe these kinds of machines in general, we need several definitions.

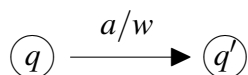
Definition 48. An **alphabet** is a finite set Σ of symbols. Given an alphabet Σ , a **string** is a finite sequence of symbols from Σ . We denote by Σ^* the set of all strings of symbols from Σ . Bear in mind that Σ includes the **empty string** λ . If $\lambda \neq w = a_1a_2 \dots a_n \in \Sigma^*$, then the **length** of w , denoted by $|w|$, is n . The length of the empty string is $|\lambda| = 0$.

For example, M_1 has $\Sigma_1 := \{P, D, q, h, R\}$ as its input alphabet and $\Sigma_2 := \{P, D, +, 0, 1, 3, 5\}$ as its output alphabet.

Definition 49. A **finite state transducer** (abbreviated *FST*) is a quintuple $M = \langle \Sigma_1, Q, s, \Sigma_2, \delta \rangle$ where

- Σ_1 is a finite set of symbols, called the **input alphabet** of M ,
- Q is a finite set, the **internal states** of M ,
- $s \in Q$ is called the **initial state** of M ,
- Σ_2 is a finite set of symbols, called the **output alphabet** of M
- $\delta : Q \times \Sigma_1 \rightarrow Q \times \Sigma_2^*$ is called the **state transition function** of M .

A finite state transducer is illustrated with a *transition diagram* such as the one in Figure 9.1. A transition diagram is a labeled directed graph whose vertices are the states of Q , and we label the arrow



from state q to state q' with a/w if $\delta(q, a) = (q', w)$, that is, in state $q \in Q$ reading input symbol $a \in \Sigma_1$ the machine is to go to state $q' \in Q$ and output string $w \in \Sigma_2^*$. When the machine gives no output, we have expressed this by writing “ $a/-$ ”. Technically this means that the output is the empty string λ , so it could also be written as “ a/λ ”.

Problem 186. Construct a transition diagram for a FST to control a stamp vending machine that asks the user to

1. choose either a 10 stamp book (\$3.40) or a 20 stamp book (\$6.80);
2. insert either \$1, \$5 or \$10 bills, larger bills before smaller bills;
3. press “RETURN” for stamps and change.

Problem 187. Construct a transition diagram for a finite state transducer that will read a string of letters from the set $\{a, b, c\}$ starting from the left (for example “caabbaccbbac”). As output, it will change each letter to an “e” until it has read three “a”s. After that, it will change each “a” to “b”, each “b” to “c” and each “c” to “a” (for example, “eeeeeeaaacbac”).

Problem 188. Construct a transition diagram for a finite state transducer which will take as input a finite string of letters from $\{a, b, c, x\}$ (for example, “abacaaccabxbxacxc”). It will output the letter “q” for each input of a, b or c until it reads an “x”. It will then divide the number of “a”s it has read by 3, return the remainder as its last output, and go back to the start state. (In the above example, the output would be “qqqqqqqqq2”.)

We can think of the input to a FST as a question and the output as the answer. For example, suppose we would like to get both a bag of peanuts (P) and a bag of Doritos (D) from the vending machine for M_1 . We could input PhhqDR to the FST to see what the vending machine would do. The FST will answer with the output P+35. This means that the vending machine would return a bag of peanuts and 35 cents change, keeping the extra 25 cents and ignoring the request for Doritos. We might then read the instructions more carefully and do this as two separate transactions.

9.2 Finite State Acceptors

Unfortunately many customers using the Peanut/Dorito M_1 machine could not follow instructions and had no access to a FST. They would insert coins before making a request, or try to make two purchases in one transaction. Often the machine would not give the intended output or would do nothing. It would get a hard kick, and would do nothing from then on. The Tater-Head-Ted’s Snack Company that designed the machine was facing bankruptcy.

Problem 189. Review the instructions 1, 2, 3 for the Peanut/Dorito M_1 machine, and make a list of all different input sequences that comply with those instructions.

Problem 190. Seeing the growing evidence of expertise in graph theory on your resumé, the Company has hired you to help. What the Company needs is a separate finite state machine M_2 that will answer the question, “Does the user’s sequence of button pushes comply with the instructions?” This machine will have no outputs, but will have a start state s and will halt in a special state y if the answer is Yes and in some other state if it is No. (For example, input “PqhR” halts in state y and input “DPhhhR” halts in some other state.) Draw a transition diagram for this finite state machine.

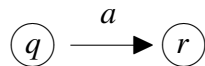
With this help, the company can now automate the checking process by incorporating both your M_2 with its M_1 into its vending machines. The new machines will save the user's input sequence in memory. When the user presses "RETURN", it will feed a copy of the input sequence to M_2 . If M_2 halts in state y , it will then feed a copy of the input sequence to M_1 . If M_2 halts in a state other than y , it will display the message, "Invalid input. Please try again!"

The finite state machine M_2 you constructed in Problem 190 is not an FST. Rather than exchanging input for output, as an FST does, its purpose is to answer the Yes-or-No question: "Did the user give a valid input as specified by the instructions?" Alternately, the question could be phrased as, "Did the user input one of the strings you found in Problem 189?" Thus M_2 is an example of a different kind of finite state machine that we will now describe.

Definition 50. A *finite state acceptor* (abbreviated as *FSA*) is a quintuple $M = (\Sigma, Q, s, Y, \delta)$ where

- Σ is a finite set of symbols, called the **input alphabet** of M ,
- Q is a finite set, the **internal states** of M ,
- $s \in Q$ is called the **initial state** of M ,
- Y is a subset of Q called the **accepting states** of M , and
- $\delta : Q \times \Sigma \rightarrow Q$ is called the **state transition function** of M .

The FSA M can be thought of as a FST that answers a Yes-or-No question. As a result, it needs no output symbols because it conveys its answer by the choice of halting state it chooses. If we interpret the input $w \in \Sigma^*$ as a question, then the answer is "Yes" if M , starting in state s , ends in a state of Y . Otherwise the answer is "No". Like a FST, the transition diagram of a FSA is a labeled directed graph. The vertices of the graph are the states of Q , and there is a labeled arrow for each state $q \in Q$ and each symbol $a \in \Sigma$. If the FSA M reads input symbol a while in state q , it will go into state $r = \delta(a, q) \in Q$ and then read the next input symbol. We illustrate this fact in the transition diagram as



We would like to extend the transition function δ to the function

$$\Delta : Q \times \Sigma^* \rightarrow Q$$

where $\Delta(q, w)$ is the state that M would reach if it were to start in state $q \in Q$ and read the string $w \in \Sigma^*$ from left to right. We can define Δ by recursion on the length of w as follows.

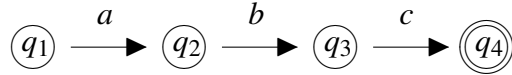
1. $\Delta(q, \lambda) = q$ and $\Delta(q, a) = \delta(q, a)$ for $a \in \Sigma$,
2. $\Delta(q, au) = \Delta(\delta(q, a), u)$ for $a \in \Sigma$ and $u \in \Sigma^*$.

For example, let $abc \in \Sigma^*$ and let $q_1 \in Q$. Define $q_2 := \delta(a, q_1)$, $q_3 := \delta(b, q_2)$ and $q_4 := \delta(c, q_3)$. If M starts in state q_1 and reads abc , it will end in state

$$\begin{aligned}\Delta(q_1, abc) &= \Delta(\delta(q, a), bc) = \Delta(q_2, bc) \\ &= \Delta(\delta(q_2, b), c) = \Delta(q_3, c) = \delta(q_3, c) = q_4.\end{aligned}$$

Definition 51. Given a FSA $M = (\Sigma, Q, s, Y, \delta)$ and a string $w \in \Sigma^*$, we say that M **accepts** the string w if $\Delta(s, w) \in Y$. We call Y the set of **accepting states**.

As we do with FSTs, we illustrate a FSA as a transition diagram with its states as the vertices. We generally label the start state as s and indicate accepting states with a double circle. If q_4 is an accepting state, the transitions described above would be shown in the transition diagram as



Definition 52. For an alphabet Σ , a **language** over Σ is a subset of Σ^* .

Definition 53. If M is a FSA, then the set

$$L = \{w \in \Sigma^* \mid \Delta(s, w) \in Y\},$$

consisting of all strings accepted by M , is called **the language recognized by M** .

For example, Problem 190 asked you to construct M_2 to recognize the language you found in Problem 189. If L is the language recognized by the FSA M , then we can think of the function of M as taking any input string $w \in \Sigma^*$, reading w from left to right and answering the question

“Are you in L ?”

In practice we often find that, after reading only part of a string, we can be certain that it will not be accepted regardless of what follows. In these cases it is convenient to specify a *dead state* which is not in Y and cannot be exited. We will use the letter “ z ” to denote a dead state. Since there may be many arrows going to z , we follow the convention that all undrawn arrows are going to z .

Example 14. Draw an FSA M_3 that recognizes the language L_3 consisting of exactly those strings that contain only letters from the input alphabet $\Sigma = \{a, b, c\}$, begin with a and end with c .

Problem 191. Draw an FSA M_4 that recognizes the language L_4 consisting of exactly those strings that contain only letters from the input alphabet $\Sigma = \{a, b, c, d\}$, begin with bc , and end with d .

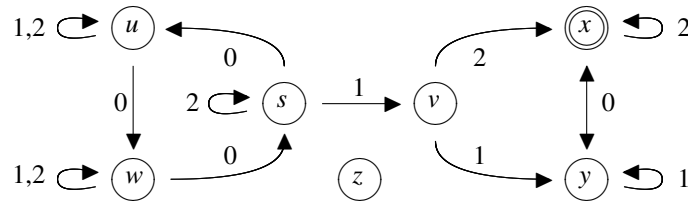


Figure 9.2: A finite state acceptor M_5 .

Problem 192. Consider the FSA M_5 illustrated in Figure 9.2. Here s is the start state, x (indicated with a double circle) is the only accepting state, and all undrawn arrows go to the ‘dead’ state z . Let $\Sigma = \{0, 1, 2\}$ and list three strings in Σ^* that are accepted by M_5 and three strings that are not accepted by M_5 .

Problem 193. Draw an FSA M_6 that recognizes the language L_6 consisting of exactly those strings that contain only the letters from the input alphabet $\Sigma = \{a, b, c\}$, begin with a , contain exactly one b and end with c .

Problem 194. Draw an FSA M_7 that recognizes the language L_7 consisting of exactly those strings that contain only the letters from the input alphabet $\Sigma = \{a, b, c, d\}$, begin with a , contain at least one b and end with cd .

Problem 195. Let $\Sigma = \{1, 2, 3, 4\}$. Construct an FSA M_8 that will answer the question, “Does $w \in \Sigma^*$ begin with 12, contain exactly four 3’s, and end with 4?”

Definition 54. If a is a character in our input alphabet and n is a positive integer, then a^n is the string consisting of a repeated n times. I.e. $a^3 = aaa$.

Problem 196. Are the commas in Problem 197 necessary?

Problem 197. Construct an FSA M_9 that recognizes the language L_9 consisting of all strings that are either

1. a positive power of C , followed by either an empty string or a string of A s and B s or
2. a positive power of D , followed by either an empty string or a string of B s and C s.

Problem 198. Let $\Sigma = \{a, b\}$. Is there an FSA that will recognize the language $L_{10} := \{a^n b^n \mid n = 1, 2, 3, \dots\}$? If so, draw it. If not, why not?

9.3 Regular Expressions

There is a compact and convenient notation to describe languages recognized by an FSA which tells, in a simple way, exactly what strings of Σ^* the FSA accepts. This notation is built recursively.

$\emptyset$ is the empty language with no strings.

λ is the one string language $\{\lambda\}$, and

a is the one string language $\{a\}$ for each $a \in \Sigma$.

If K and L are each a language, then

$KL = \{uv \mid u \in K \text{ and } v \in L\}$, the **concatenation** of K and L ,

$K \cup L = \{w \in \Sigma^* \mid w \in K \text{ or } w \in L\}$, the **union** of K and L ,

$L^* = \{\lambda\} \cup \{w \in \Sigma^* \mid w = u_1 u_2 \dots u_n \text{ for some } u_1, u_2, \dots, u_n \in L\}$, the **Kleene star** of L .

For example, consider the language L_4 described in Problem 191. A string in L_4 looks like

$bc \dots \text{any string} \dots d$.

This is the concatenation of three languages: $\{bc\}$, all strings (i.e., Σ^*) and $\{d\}$. To express each of these as a regular expression, we can use $\mathbf{b} = \{b\}$, $\mathbf{c} = \{c\}$ and $\mathbf{d} = \{d\}$. Then we have the concatenation $\mathbf{bc} = \{b\}\{c\} = \{bc\}$. In this example we have $\Sigma = \{a, b, c, d\} = \mathbf{a} \cup \mathbf{b} \cup \mathbf{c} \cup \mathbf{d}$. Notice that the set of all strings from Σ , that we have called Σ^* , is in fact an example of the Kleene star. Thus we have $\Sigma^* = \{a, b, c, d\}^* = (\mathbf{a} \cup \mathbf{b} \cup \mathbf{c} \cup \mathbf{d})^*$. The language L_4 can now be described by the regular expression that is the concatenation of these three:

$$L_4 = \{bc\}\Sigma^*\{d\} = \mathbf{bc}(\mathbf{a} \cup \mathbf{b} \cup \mathbf{c} \cup \mathbf{d})^*\mathbf{d}.$$

Problem 199. Find a regular expression for the language L_3 described in Example 14.

Problem 200. Find a regular expression for the language L_5 , where M_5 is the FSA drawn in Figure 9.2.

Problem 201. Find a regular expression for the language L_6 described in Problem 193.

Problem 202. Find a regular expression for the language L_7 described in Problem 194.

Problem 203. Find a regular expression for the language L_8 described in Problem 195.

Problem 204. Find a regular expression for the language L_9 described in Problem 197.

Problem 205. Draw a transition diagram for an FSA that recognizes the language $(aba)^*(ddd)(b \cup e)^*$.

Problem 206. Consider FSAs $M = \langle \Sigma, Q, s, Y, \delta \rangle$ recognizing language L and $M' = \langle \Sigma, Q', s', Y', \delta' \rangle$ recognizing language L' . Tell how to use M and M' to design an FSA $M^\cup = \langle \Sigma, Q^\cup, s^\cup, Y^\cup, \delta^\cup \rangle$ that will recognize $L \cup L'$. (HINT: Try using $Q^\cup = Q \times Q'$ for the states of $M^\cup$, and then define $s^\cup$, $Y^\cup$ and $\delta^\cup$.)

The direct product construction used in Problem 206 for $Q^\cup$ tends to produce an FSA with an excessively large number of states. In practice most of these states are unnecessary since they can not be reached from the start state. A practical way to construct the transition diagram for the FSA M in Problem 206 is to begin with the start state and add in only those states that can be reached by successive transitions from the start state. Try this in the following extension of Problem 205.

Problem 207. Draw a transition diagram for an FSA that recognizes the language

$$(aba)^*(ddd)(b \cup e)^* \cup a^*(d^* \cup c^*).$$

These problems suggest an important theorem about languages which says that a language is recognized by some finite state acceptor if and only if it can be represented by a regular expression.

9.4 Project: Reversed Strings and Palindromes

Definition 55. If $w \in \Sigma^*$, we define $w^R \in \Sigma^*$ recursively as follows.

1. $\lambda^R = \lambda$ and $a^R = a$ for all $a \in \Sigma$.
2. If $w = av$ where $a \in \Sigma$ and $v \in \Sigma^*$, then $w^R = v^R a$.

Problem 208. Use Definition 55 to compute w^R if $w = pots$. What is w^R if $w = scitamehtametercsid$?

Problem 209. Let $\Sigma = \{a, b\}$. List all the strings in the finite language $L = \{w \in \Sigma^* \mid w = uu^R u \text{ for some } u \in \Sigma^*\}$, remembering that $\Sigma\Sigma$ is the concatenation of Σ with itself.

Problem 210. Let $\Sigma = \{a, b\}$. Describe a method to generate all of the strings in the language $L = \{w \in \Sigma^* \mid www = uu \text{ for some } u \in \Sigma^*\}$, and explain why your method is correct.

Problem 211. Let $\Sigma = \{a, b\}$ and $L = \{w \in \Sigma^* \mid w^R w = w w^R\}$. Give examples of two strings in L and two strings not in L . Describe the strings of L in English.

Definition 56. A string $w \in \Sigma^*$ is a **palindrome** if $w^R = w$.

For example, A, EYE, NOON, CIVIC and MADAM are all palindromes.

Problem 212. Prove by induction on the length $|u|$ of the string $u \in \Sigma^*$ that, for all $v \in \Sigma^*$, we have $(uv)^R = v^R u^R$.

Problem 213. Let $u \in \Sigma^*$ and $b \in \Sigma$. Show that uu^R and ubu^R are both palindromes.

In fact, these are the only palindromes!

Problem 214. Let $w \in \Sigma^*$.

- (i) If $|w|$ is even, show that w is a palindrome if and only if there is a $u \in \Sigma^*$ such that $w = uu^R$.
- (ii) If $|w|$ is odd, show that w is a palindrome if and only if there are a $b \in \Sigma$ and a $u \in \Sigma^*$ such that $w = ubu^R$.

Problem 215. Below is a list of palindromes. Referring to Problem 214 tell, for each one, what u is if the palindrome has even length and what u and b are if it has odd length.

I PREFER PI

A TOYOTAS A TOYOTA

DO GEESE SEE GOD

WAS IT A CAR OR A CAT I SAW

AL LETS DELLA CALL ED STELLA

RATS LIVE ON NO EVIL STAR

DID HANNAH SEE BEES HANNAH DID

IN WORD SALAD ALAS DROWN I

Chapter 10

Machine Minimization

When writing a computer program or mathematical proof, the first goal is to create one that works. The second goal is to make it as simple as possible. The simpler a program or proof is, the easier it is to be certain that it really does work. The simpler it is, the easier it is to maintain and modify.

The finite state acceptor model for computing machines gives us a context in which this simplification can be fully realized. In this chapter we will see how, given a finite state acceptor, we can construct a new finite state acceptor of minimal complexity that recognizes exactly the same language. This work will lead to a very useful test to tell whether or not a given language is recognized by some FSA.

In general it is difficult to measure the complexity of a computer program or mathematical proof. A short program or proof can utilize complex logic while a long program or proof can be straightforward and even repetitive. In the case of a finite state acceptor $M = \langle \Sigma, Q, s, \delta, Y \rangle$, the **number of internal states**, $|Q|$, provides us with a useful measure of the complexity of M . Our goal is to find a method to generate from a FSA M recognizing a language L , a new FSA M' such that

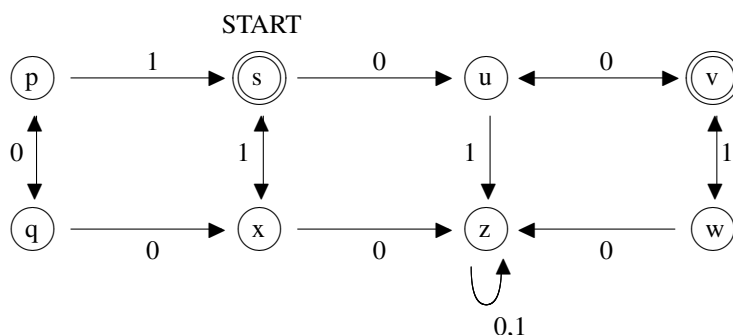
1. M' also recognizes L and
2. no other FSA that recognizes L has fewer states than M' .

Such a FSA M' is called a **minimal** FSA. To see an example of a minimal FSA, let L_0 be the language recognized by the FSA M_0 in Figure 10.1.

Problem 216. Note that M_0 has two accepting states, s and v . Find a regular expression for the language L_0 .

Problem 217. Construct a FSA M'_0 that also recognizes L_0 but has only 4 states, one of which will be a dead state.

Problem 218. Let N be any FSA that recognizes L_0 and let s' be its start state. Explain why s' , $\delta(s', 0)$, $\delta(s', 1)$ and $\Delta(s', 10) = \delta(\delta(s', 1), 0)$ must all be distinct states.

Figure 10.1: A Finite State Acceptor, M_0 .

You have just shown that any FSA N that recognizes L_0 has at least as many states as M'_0 that you constructed in Problem 217. Thus your FSA M'_0 is a minimal FSA for M_0 in Figure 10.1. Our goal is to find a general procedure that will produce, from any FSA, a minimal FSA that recognizes the same language. Let Σ be any alphabet.

Definition 57. Given strings $u, v \in \Sigma^*$, the **concatenation** of u and v is the string uv formed by tacking v onto the end of u . Note that λ is an identity relative to the binary operation of concatenation, as we have $\lambda w = w = w\lambda$ for all $w \in \Sigma^*$.

Definition 58. Let $L \subseteq \Sigma^*$ be a language and $w \in \Sigma^*$. We define the **right set** for w to be $R_w = \{z \in \Sigma^* \mid wz \in L\}$.

Problem 219. Using the regular expression you found in Problem 216 for L_0 , find a regular expression for each of these right sets.

$R_0 \quad R_{001} \quad R_{00110} \quad R_{0011011} \quad R_{10} \quad R_{11} \quad R_{11000111} \quad R_{0011}$

Definition 59. Let $L \subseteq \Sigma^*$ be a language. If $x, y \in \Sigma^*$, then we write $x \equiv_L y$ if $R_x = R_y$.

Problem 220. Show that if $L \subseteq \Sigma^*$ is any language, then $\equiv_L$ is an equivalence relation on Σ^* .

Note that no string in L begins with w if and only if $R_w = \emptyset$. Thus the set of all strings that are not a prefix of any member of L is an $\equiv_L$ -class. We denote this $\equiv_L$ -class by $Z(L)$, the set of strings whose right set is empty.

Problem 221. Referring to Problem 219, tell which of the strings $0, 001, 00110, 0011011, 10, 11, 11000111, 0011$ are $\equiv_{L_0}$ -equivalent.

Problem 222. Let $T = \{\lambda, 0, 1, 01\}$.

1. Show that no two strings in T are $\equiv_{L_0}$ -equivalent.

2. Explain why every string $w \in \{0, 1\}^*$ is $\equiv_{L_0}$ -equivalent to a string in T .

If $w \in \{0, 1\}^*$, let $[w]$ denote the $\equiv_0$ -class of w . Problem 222 shows that there are exactly four different $\equiv_{L_0}$ -classes:

$$[\lambda] = L_0 \quad [0] = L_0 0 \quad [1] = L_0 1 \quad [01] = \Sigma^* \sim (L_0 \cup L_0 0 \cup L_0 1) = Z(L_0).$$

The arrows in Figure 10.2 show where we go if we extend a string in any one of these classes by either 0 or 1.

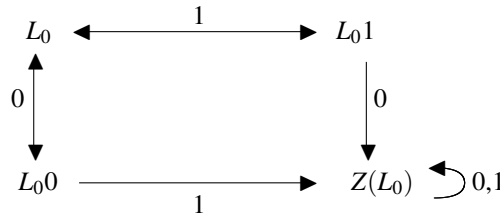


Figure 10.2: M_0'' , A Minimal Finite State Acceptor for L_0

Since $\lambda \in L_0$, we can take any string $w \in \{0, 1\}^*$ and find which $\equiv_{L_0}$ -class w is in by tracing the diagram, building w by starting with λ in L_0 and successively adding 0s or 1s on the right until we have w . Our ending point will be the $\equiv_{L_0}$ -class of w . If we land in L_0 , then $w \in L_0$; otherwise $w \notin L_0$. What we have done is to find a systematic way to build an FSA M_0'' for the language L_0 using the $\equiv_{L_0}$ -classes as internal states and taking L_0 as the only accepting state. (See Figure 10.2.) In fact, according to Problem 218, M_0'' is a *minimal* FSA to recognize L_0 !

We would like to mimic this construction, starting with an arbitrary language $L \subseteq \Sigma^*$ to see if we can construct a FSA for L with a minimum number of states. In order to do this, it will be helpful to start with a broader setting in which we make no restrictions on the number of states in Q .

Definition 60. A *General State Acceptor* (abbreviated as *GSA*) is a quintuple $G = \langle \Sigma, Q, s, \delta, Y \rangle$ defined exactly as we defined a FSA but without the restriction that the set Q of states must be finite.

While it is not possible to build a GSA with infinitely many states, it will be helpful here to look first for a GSA for L and then see when we can find one with only finitely many states. Following the construction of M_0'' , we define

$$G_L := \langle \Sigma, Q_L, s_L, \delta_L, Y_L \rangle$$

by taking

$$\begin{aligned} Q_L &:= \{[w] \mid w \in \Sigma^*\} & s_L &:= [\lambda] \\ Y_L &:= \{[w] \mid w \in L\} & \delta_L([w], a) &:= [wa]. \end{aligned}$$

Problem 223. Let Σ be an alphabet and let $L \subseteq \Sigma^*$ be a language. Then, for all $u, v \in \Sigma^*$ and $a \in \Sigma$,

1. $[u] \in Y_L$ if and only if $u \in L$,
2. $[u] = [v]$ implies $[ua] = [va]$.

Note that Problem 223(2) implies that δ_L is well defined.

Recall that Δ_L is the extension of δ_L to all of $Q_L \times \Sigma^*$.

Problem 224. Use mathematical induction on the length of v to show that, for all $u, v \in \Sigma^*$,

$$\Delta_L([u], v) = [uv].$$

Problem 225. Use Problems 223(2) and 224 to show that G_L recognizes the language L .

This shows that every language $L \subseteq \Sigma^*$ is recognized by some GSA. The GSA G_L is called the **standard GSA** for L .

Example 15. Let $\Sigma = \{a, b\}$ and construct the GSA $G_{L_{10}}$ for the language $L_{10} = \{a^n b^n \mid n = 1, 2, 3, \dots\}$ of Problem 198 in the following steps.

1. Show that the $\equiv_{L_{10}}$ -classes $[b], [\lambda], [a], [a^2], [a^3], [a^4], \dots$ are all different, and that consequently $Q_{L_{10}}$ is infinite.
2. Show that every string is $\equiv_{L_{10}}$ -equivalent to one of the strings in the list $b, \lambda, a, a^2, a^3, a^4, \dots$. Thus $Q_{L_{10}} = \{[b], [\lambda], [a], [a^2], [a^3], [a^4], \dots\}$ is the set of all states of $G_{L_{10}}$.
3. Figure 10.3 gives a skeleton of the transition diagram for $G_{L_{10}}$. Complete this diagram by assigning one of the states of $Q_{L_{10}}$ to each of its states, identifying the start state and the accepting states.
4. Explain from your diagram why $G_{L_{10}}$ accepts exactly the strings in L_{10} .

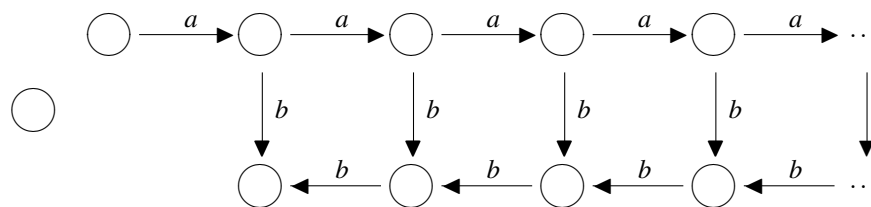


Figure 10.3: Transition diagram for a GSA with infinitely many states.

Example 15 shows how GSAs with infinitely many states can naturally arise from simply defined languages. But our real interest is in FSAs, which

might actually be built. Do you think that there could be an FSA that recognizes L_{10} ? We can now exhibit a special feature of standard GSAs which at last proves for certain that there is not.

Problem 226. *Show that if L is a language, then every GSA that recognizes L has at least as many states as the standard GSA for L .*

Hint. Let $M = \langle \Sigma, Q, s, \delta, Y \rangle$ be any GSA that recognizes L . Consider a fixed list $w_1, w_2, w_3, \dots$ of strings in Σ^* , one from each $\equiv_L$ -class. Use Problem 224 to show that $\Delta(s, w_1), \Delta(s, w_2), \Delta(s, w_3), \dots$ is a list of distinct states of M , one for each of the states of Q_L .

Given a language $L \subseteq \Sigma^*$, we can now determine whether or not it is recognized by some FSA and, if it is, construct a minimal FSA that recognizes it. All we need to do is to calculate right sets to determine the $\equiv_L$ -classes of Q_L . We state what we have found as a final theorem.

FSA Theorem. *Let $L \subseteq \Sigma^*$ be a language with Q_L the set of all $\equiv_L$ -classes.*

1. *If Q_L is finite, then the standard GSA for L is a minimal FSA that recognizes L .*
2. *If Q_L is infinite, then no FSA recognizes L .*

Example 16. *The standard FSA for $L_0 = (\mathbf{00} \cup \mathbf{11})^*$ is a minimal FSA that recognizes L_0 .*

To see this, let $w \in \{0, 1\}^*$.

1. If $w \in (\mathbf{00} \cup \mathbf{11})^*$, then its right set is $(\mathbf{00} \cup \mathbf{11})^*$.
2. If $w \in (\mathbf{00} \cup \mathbf{11})^* \mathbf{0}$, then its right set is $\mathbf{0}(\mathbf{00} \cup \mathbf{11})^*$.
3. If $w \in (\mathbf{00} \cup \mathbf{11})^* \mathbf{1}$, then its right set is $\mathbf{1}(\mathbf{00} \cup \mathbf{11})^*$.
4. Otherwise its right set is $\emptyset$.

Consequently there are only four different $\equiv_{L_0}$ -classes in Q_{L_0} .

Example 17. *The language $L_{10} = \{a^n b^n \mid n = 1, 2, 3, \dots\}$ is not recognized by any FSA.*

To see this, consider the infinite set $\{a, a^2, a^3, a^4, \dots\}$ of strings in $\{a, b\}^*$. For each positive integer n , the string b^n is in the right set of a^n but not in the right set of a^m for any $m \neq n$. Consequently $Q_{L_{10}}$ has infinitely many different $\equiv_{L_{10}}$ -classes.

Show that Q_L is finite for each of the following languages L , and then construct its standard FSA.

Problem 227. $L = 0^*1^*2^*$

Problem 228. $L = 00(1^* \cup 2^*)$

Problem 229. $L = (1^* \cup 2^*)00$

Problem 230. $L = 111 \cup (020)^*$

Problem 231. $L = (111)^*1$

For the next four problems, demonstrate that Q_L is infinite for the given language L , showing that L is not recognized by any FSA.

Problem 232. $L = \{0^n10^n \mid n = 1, 2, 3, \dots\}$

Problem 233. $L = \{wcw \mid w \in \{a, b\}^*\}$

Problem 234. L_{re} is the set of regular expressions over the alphabet $\{a, b, c\}$.

Problem 235. $L \subseteq \{a, b\}^*$ is the set of strings w containing the same number of a 's as b 's.

Problem 236. In 1931 the famous logician Kurt Gödel proved that there would never be a computing machine that could correctly tell us (Yes-or-No) if an arbitrary statement about positive integers is true. Prove the special case of this theorem for FSAs: "There is no FSA that can correctly tell us (Yes-or-No) if an arbitrary statement about the positive integers is true."